

Введение в язык программирования C#

Описание языка C#

Язык программирования C# является объектно-ориентированным языком программирования высокого уровня. Автором языка является Андерс Хейлсберг, который был архитектором Turbo Pascal и Borland Delphi.

Язык C# относится к C-подобным языкам программирования по синтаксису.

Изначально язык C# был разработкой для работы на прикладном уровне с CLR (Common Language Runtime) – среда, компилирующая программы, которые написаны на языках .NET (Visual C#, Visual Basic, Visual F#, Visual C++ и др.)

C# стандартизирован в ECMA (ECMA-334) и ISO (ISO/ IEC 23270).

Типы данных языка C#

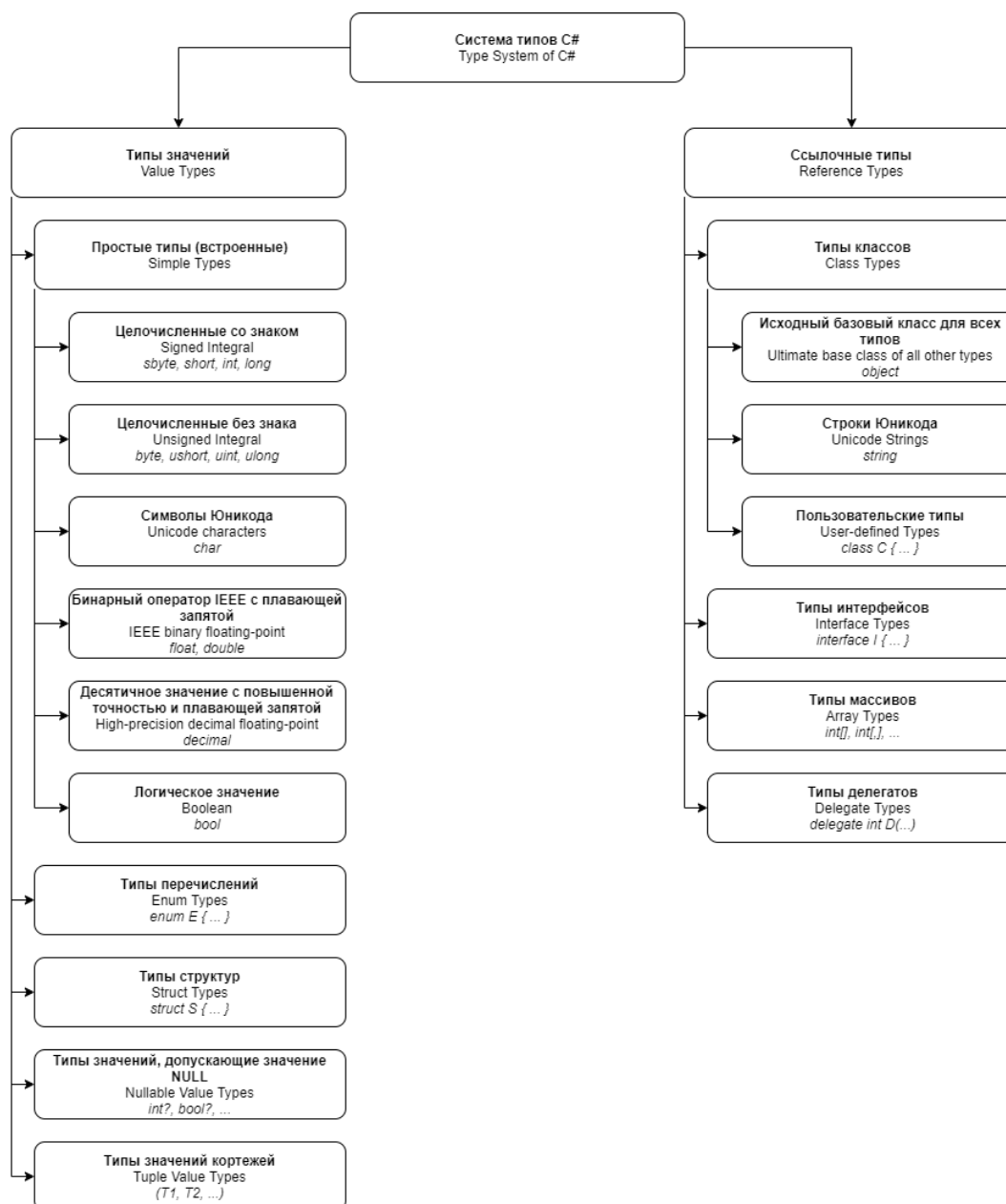


Рисунок 1 - Система типов в языке C#

Типы данных языка программирования C# разделяются на две разновидности: типы значений [знаковые] (переменные таких типов хранят в себе значения) и ссылочные типы (в переменной такого типа будет храниться ссылка на объекты [данные]).

Структура программы

```
using System; // Подключение библиотек
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;
using static System.Console; // Доступ к статическим членам и вложенным типам

namespace _01_Structure // Пространство имен (Название проекта)
{
    class Program // Описание класса
    {
        static void Main(string[] args) // Метод Main
        {
            WriteLine("Hi, STANKIN!");
            ReadLine();

            Console.WriteLine("Программисты тут все чтоли?");
            Console.ReadLine();

            string lines = "Строка 1 \nСтрока 2\nСтрока 3";

            using (var reader = new StringReader(lines)) // Оператор using (statement)
            {
                var item = reader.ReadLine();
                while (item != null)
                {
                    Console.WriteLine(item);
                    Thread.Sleep(3000);
                    item = reader.ReadLine();
                }
            }
            Console.ReadLine();
        }
    }
}
```

Using. Инструкция и директива

Ключевое слово using используется, как правило, в трех случаях:

- Необходимо подключить библиотеку (directive using). Действует до конца файла.
- Необходимо получить доступ к статическим членам и вложенным типам без указания их имен (directive using static). Действует до конца файла.
- Необходимо получить один или несколько ресурсов, выполнить инструкции и уничтожить ресурс (using statement) – оператор using. Действует до закрывающейся фигурной скобки (до версии языка C# 8.0)

Директива #region

В случае необходимости создания удобочитаемости кода в виде сворачивания и разворачивания его элементов в редакторе кода, можно использовать директиву #region.

```
#region NameOfRegion
// Код
#endregion
```

Тип данных Object. Упаковка и распаковка [Boxing-Unboxing]

Все типы данных в языке C# (в том числе встроенные и написанные самостоятельно) являются неявными наследниками класса System.Object. Именно поэтому, все классы, написанные самостоятельно уже имеют методы, которые определены в классе Object (ToString(), GetHashCode(), ...)

Упаковка [Boxing] – процесс неявного преобразования типа значения в тип класса Object.

```
int mark = 25; // в переменной mark лежит значение 25
object markObject = mark;
// неявное преобразование типа значения int в ссылочный тип object
```

Распаковка [Unboxing] – процесс извлечения типа значения из объекта (явное преобразование). При данной процедуре необходимо проверить экземпляр объекта на то, что он был упакован значением заданного типа данных, а уже после применяется процедура копирования значения в переменную типа значения.

```
object valueObject = 54;
int value = (int)valueObject;
// явное преобразование в тип значения int из ссылочного типа object
```

Процедура упаковки-распаковки может потребоваться в том случае, если необходимо избежать повторяющегося кода или смешать объекты разных типов в одной коллекции. Стоит отметить, что Boxing-Unboxing – это довольно медленные операции.

```
object[] trash = new object[3];
trash[0] = 24; // упаковка int
trash[1] = -54.01m; // упаковка decimal
trash[2] = "Когда экзамен?";
// в массиве типа object могут быть и объекты ссылочного типа
```

Тип данных Struct

Тип структуры является типом значения (переменная такого типа содержит экземпляр этого типа). В структуре могут содержаться элементы-данные и элементы-функции.

```
public struct Person
{
    string _name; // private field (приватное поле)
    string _lastName; // private field (приватное поле)

    /// <summary>
    /// Constructor. Конструктор
    /// </summary>
    /// <param name="name">Name of Person. Имя</param>
    /// <param name="lastName">Last Name of Person. Фамилия</param>
    public Person(string name, string lastName)
    {
        _name = name;
        _lastName = lastName;
    }

    /// <summary>
    /// Get Full Name. Получить полное имя
    /// </summary>
```

```
/// <returns>Full Name (Name + Last Name). Полное имя</returns>
public string GetFullName()
{
    return $"Name: {_name}; Last Name: {_lastName}";
}

class Program
{
    static void Main(string[] args)
    {
        Person lecturer = new Person("Nikita", "Kashurkin"); // instance (представитель
структуры)
        Console.WriteLine(lecturer.GetFullName()); // method call (вызов метода)
    }
}
```

Все элементы, инкапсулированные внутри структуры, по умолчанию имеют модификатор доступа `private`. Поэтому нет необходимости писать этот модификатор доступа перед объявлением локальной переменной.

Наследование для структур

Так как структура относится к типу значений, она не может выступать членом наследования (ни базовым, ни наследуемым), но она может реализовывать интерфейс.

Поля структуры

При описании поля в структуре нельзя его одновременно инициализировать, если оно не является статическим.

Конструктор структуры

Если в структуре не описан конструктор, будет автоматически создан конструктор по умолчанию без параметров. При описании структуры нет возможности самостоятельно реализовать конструктор без параметров. Более того, в конструкторе должны инициализироваться все поля структуры.

Деструктор структуры

При реализации структуры нельзя объявить деструктор.

Что лучше использовать: структуру или класс?

<https://docs.microsoft.com/en-us/dotnet/standard/design-guidelines/choosing-between-class-and-struct>

Массивы

В языке C# массив представляет собой указатель на непрерывный участок памяти. Другими словами, на этом языке имеются только динамические массивы.

Массив рассматривается, как класс, что позволяет пользоваться его методами и свойствами.

Объявление одномерного массива

```
DataType[] name; // Тип_данных[] наименование_массива;
```

Объявление многомерного массива

```
DataType[, ] name; // Тип_данных[, ] наименование_массива;
```

Перед использованием массива он должен быть инициализирован, то есть под него должна быть выделена память.

Одномерные массивы

```
int[] myArray; // Объявление массива
myArray = new int[5]; // Выделение памяти под массив на 5 элементов
```

```
// Заполнение массива
for (var i = 0; i < myArray.Length; i++)
{
    Console.WriteLine($"myArray[{i}] = ");
    myArray[i] = Convert.ToInt32(Console.ReadLine());
}

int sum = 0;
foreach (var item in myArray) // Оператор foreach
{
    sum += item;
}

// Вывод массива
for (var i = 0; i < myArray.GetLength(0); i++)
{
    Console.WriteLine($"myArray[{i}] = {myArray[i]}");
}

Console.WriteLine($"Sum = {sum}");
Console.ReadLine();
```

Массив является ССЫЛОЧНЫМИ типом данных.

```
// Копирование массивов
int[] secondArray = myArray;
secondArray[1] = myArray[1] - 200;

Console.WriteLine("myArray:");
PrintArray(myArray);
Console.WriteLine("secondArray:");
PrintArray(secondArray);
Console.ReadLine();

int[] thirdArray = new int[secondArray.Length];
secondArray.CopyTo(thirdArray, 0); // Корректное копирование массивов

thirdArray[1] = secondArray[1] - 200;

Console.WriteLine("thirdArray:");
PrintArray(thirdArray);
Console.ReadLine();
```

Многомерные массивы:

```
decimal[,] array = new decimal[4, 5];
Random random = new Random();

for (var i = 0; i < array.GetLength(0); i++)
    for (var j = 0; j < array.GetLength(1); j++)
    {
        array[i, j] = Math.Round((random.Next(1050, 3000) / 3.0m), 2);
    }

for (var i = 0; i < array.GetLength(0); i++)
{
    for (var j = 0; j < array.GetLength(1); j++)
    {
        Console.Write(array[i, j] + "\t");
    }
    Console.WriteLine();
}
Console.ReadLine();
```

Рваные (ступенчатые) массивы.

Массив массивов (Jagged Array). Вид массива, элементами которого могут являться массивы разных размерностей. Такие массивы также называют массивами массивов.

```
int[][] array = new int[3][]; // declaration

array[0] = new int[3];
array[0][0] = 11; // initialization
array[0][1] = 12;
array[0][2] = 13;

array[1] = new int[5] { 21, 22, 23, 24, 25 }; // initialization

array[2] = new int[] { 31, 32 }; // initialization

for (var i = 0; i < array.Length; i++)
{
    for(var j = 0; j < array[i].Length; j++)
    {
        Console.Write($"[{i}][{j}] = {array[i][j]}\t");
    }
    Console.WriteLine();
}
Console.WriteLine();

for (var i = 0; i < array.Length; i++)
{
    foreach(var item in array[i])
    {
        Console.Write($"{item}\t");
    }
    Console.WriteLine();
}
Console.ReadLine();
```

Коллекции

Для эффективного управления и обработки данных рекомендуется организовывать коллекции. Типы коллекций – это виды коллекций данных, таких как хэш-таблицы, очереди, стеки, контейнеры, словари и списки.

Список List<T>

Тип данных «Список» (List<T>), реализованный в C#, представляет собой строго типизированный список объектов, доступных по индексу. Поддерживает методы для поиска по списку, выполнения сортировки и других операций со списками.

Полный перечень методов для работы со списками можно получить, если ввести название списка и поставить точку. Среда программирования сама выведет данный перечень доступных методов.

Списки, закрытые встроенным типом данных

```
List<int> myList = new List<int>(); // Список, закрытый встроенным типом данных int
Random random = new Random();

// Наполнение списка
for (var i = 0; i < 10; i++)
{
    myList.Add(random.Next(-100, 100)); // Добавление элемента в конец списка
}

Console.WriteLine("Заполненный список:");
foreach (var item in myList)
{
    Console.WriteLine(item);
}
```

```
}

Console.WriteLine($"Максимальный элемент списка равен: {myList.Max()}");
Console.WriteLine("Отсортированный список:");
foreach (var item in myList.OrderByDescending(x => x))
{
    Console.WriteLine(item);
}
Console.WriteLine($"Сумма элементов списка: {myList.Sum()}");

Console.WriteLine("Неотрицательные элементы списка:");
foreach (var item in myList.Where(el => el >= 0))
{
    Console.WriteLine(item);
}

Console.ReadLine();
```

Списки, основанные на написанных программистом типах

```
List<Person> students = new List<Person>(); // Произвольный тип Person

var student1 = new Person("Владислав", "Моисеев");
students.Add(student1);

students.Add(new Person("Денис", "Комарков"));
students.Add(new Person() { Name = "Никита", LastName = "Паничев" });

var student4 = new Person();
Console.Write("Введи фамилию:");
student4.LastName = Console.ReadLine();
Console.Write("Введи имя:");
student4.Name = Console.ReadLine();

students.Add(student4);

Console.WriteLine("Студенты:");
students.ForEach(s => Console.WriteLine(s.GetFullName()));

Console.ReadLine();
```

Словарь Dictionary<TKey, TValue>

В некоторых случаях может потребоваться использование пары «ключ-значение». В том случае, если в качестве ключа выступает обычное значение `int`, можно использовать обычный массив. Но иногда в качестве ключа необходимо использование значения какого-то нестандартного типа. В данном случае можно использовать тип `Dictionary<TKey, TValue>`, где и в качестве ключа, и в качестве значения можно использовать любые типы данных.

Создание словаря аналогично пройденному ранее созданию списка. Добавление в словарь происходит при помощи метода `Add(TKey, TValue)`.

```
var lecturers = new Dictionary<string, string>();

lecturers.Add("Программирование", "Носовицкий В.Б.");
lecturers.Add("МатАн", "Петросян Н.С.");
//lecturers.Add("Программирование", "Шибеева А.Н."); // Ошибка
lecturers.Add("СиАОД", "Лакунина О.Н.");
lecturers.Add("ООП", "Кашуркин Н.");

foreach (var lect in lecturers)
    Console.WriteLine(lect);
Console.Write("Введи ключ для поиска:");
var key = Console.ReadLine();
string lecturer;
// Обращение по ключу
try
{
    // В случае, если элемент не будет найден, будет сгенерирована исключительная ситуация
    lecturer = lecturers[key];
}
catch
{
    lecturer = "Не найдено";
}
Console.WriteLine($"Результат обращения по ключу: {lecturer}");

// Поиск по ключу
var lecturer2 = lecturers.FirstOrDefault(l => l.Key.ToLower() == key.ToLower()).Value
?? "Не найдено";
Console.WriteLine($"Результат поиска по ключу: {lecturer2}");

if(lecturer != "Не найдено")
{
    Console.Write($"Введите новое значение для lecturer[{key}]: ");
    lecturers[key] = Console.ReadLine();

    foreach (var lect in lecturers)
        Console.WriteLine(lect);

    Console.WriteLine($"Удаление элемента с ключом {key}...");
    lecturers.Remove(key);
    foreach (var lect in lecturers)
        Console.WriteLine(lect);
}
```

```
[Программирование, Носовицкий В.Б.]
[МатАн, Петросян Н.С.]
[СиАОД, Лакунина О.Н.]
[ООП, Кашуркин Н.]
Введи ключ для поиска:МатАн
Результат обращения по ключу: Петросян Н.С.
Результат поиска по ключу: Петросян Н.С.
Введите новое значение для lecturer[МатАн]: Уварова Л.А.
[Программирование, Носовицкий В.Б.]
[МатАн, Уварова Л.А.]
[СиАОД, Лакунина О.Н.]
[ООП, Кашуркин Н.]
Удаление элемента с ключом МатАн...
[Программирование, Носовицкий В.Б.]
[СиАОД, Лакунина О.Н.]
[ООП, Кашуркин Н.]
```

Тип перечислений Enum

Тип перечисления – такой тип данных, который определяется набором перечисленных по имени констант. Enum относится к типам по значению (как структура).

```
public enum Имя
{
    Значение_1,
    Значение_2,
    Значение_3
}
```

Все значения констант по умолчанию имеют тип данных `int`, но вовсе не обязательно использовать именно его. Если необходимо использовать другой целочисленный тип, можно поставить после имени знак «:» и указать необходимый тип.

Все порядковые номера в энумераторе присваиваются значениям, начиная с единицы.

Программист вправе сам решить значения для своего энумератора.

```
public enum MonthEnum
{
    January = 1,
    February = 2,
    March = 3,
    April = 4,
    May = 5,
    June = 6,
    July = 7,
    August = 8,
    September = 9,
    October = 10,
    November = 11,
    December = 12
}
```

Для использования энумератора необходимо вызвать его с синтаксисом: «Имя.Значение_1». Для получения значения энумератора можно использовать приведение типов `((int)Имя.Значение_1)`.

```
Console.WriteLine($"MonthEnum.January : {MonthEnum.January}"); // January
Console.WriteLine($"(int)MonthEnum.January : {(int)MonthEnum.January}"); // 1
```

Объектно-Ориентированное Программирование

Базовые понятия ООП

Базовыми понятиями объектно-ориентированного программирования являются объект и класс. Объект – это какой-то реально существующий предмет со всеми его индивидуальными характеристиками. Класс – это множество объектов с одинаковыми характеристиками и одинаковым поведением. При определении значений характеристик класс превращается в объект. Характеристики класса задают данными, а поведение – методами. В С# методы представляют собой функции; среди методов выделяют конструктор и финализатор – функции особого назначения и с особыми правилами оформления.

Основные принципы ООП

Основными принципами объектно-ориентированного программирования являются:

- Абстракция
- Инкапсуляция
- Наследование
- Полиморфизм

Абстракция – выделение наиболее важных в рамках решения определенной задачи характеристик объекта. Например, необходимо написать программу по работе со студентами. В классе «Студент» будут описаны такие данные, как номер студенческого билета, фамилия, имя, дата рождения, место жительства и так далее. В рамках решения поставленной задачи мы абстрагируемся от таких характеристик, например, как рост и вес. Инкапсуляция – объединение в одной структуре данных (структуре или классе) объявления данных и методов их обработки.

Наследование – возможность получать из одного класса другой с сохранением необходимых элементов в дочерних классах, модифицируя их, добавляя при необходимости новые возможности.

Полиморфизм – возможность иметь несколько реализаций одного элемента с автоматическим выбором подходящего.

Модификаторы доступа в языке С#

Модификаторы доступа позволяют определить доступность тех или иных элементов в структуре программы. В языке С# есть несколько модификаторов доступа:

- `public` – общедоступный элемент;
- `private` – закрытый элемент (элемент, помеченный таким модификатором доступа, доступен только в том элементе, в котором он объявлен);
- `protected` – элемент, доступный внутри того класса, в котором он объявлен, а так же в элементах, наследуемых от этого класса;
- `internal` – элемент, доступный для использования внутри сборки (проекта), в котором он указан;
- `protected internal` – элемент, доступный из текущей сборки и в наследниках от класса, в котором он объявлен;
- `private protected` – элемент, доступный внутри класса, в котором он объявлен, или в производных классах, которые определены в той же сборке.

Стоит отметить, что в языке С# в классах и структурах элементы по умолчанию имеют модификатор доступа `private`. Классы, написанные без модификатора доступа, по

умолчанию имеют модификатор доступа `internal`. Стоит также отметить, что действие модификатора доступа продолжается до знака «;».

```
class A // = internal class A
{
    int a; // Поле, доступное только внутри класса
    private int b; // Поле, доступное только внутри класса
    protected decimal c; // Поле, доступное внутри класса и классах-наследниках от
    класса A
    public string d; // Поле, доступное в любом месте кода

    internal double e; // Поле, доступное в любом месте кода этой сборки

    protected internal int f; // Поле, доступное в любом месте кода этой сборки и в
    классах-наследниках от A
    private protected char g; // Поле, доступное внутри класса A и в классах-
    наследниках от A в этой сборке
}
```

Благодаря механизму модификаторов доступа можно скрывать некоторые методы и свойства класса от других частей программы, что и является инкапсуляцией.

Классы. Элементы класса

Класс предоставляет возможность программисту разрабатывать свои собственные типы данных путем группирования других типов и методов. Основным отличием классов от структур является возможность наследования.

Элементы класса:

- Поле. Переменная, инкапсулированная в классе. Как правило, закрытая.
- Свойство. Механизм для чтения, записи или вычисления значения элемента класса (например, скрытого). Позволяет легко получать доступ к данным и помогает повысить безопасность и гибкость методов. Метод доступа `get` используется для возврата значения свойства, а метод доступа `set` – для присвоения нового значения. Эти методы могут иметь различные уровни доступа. Параметр `value` используется в методе `set` для определения значения свойства и имеет тот же тип, что и свойство. Если `set` не реализован, то свойство доступно только для чтения.
- Метод. Механизм, позволяющий обрабатывать данные и возвращать результат работы над данными, если необходимо.
- Специальные методы и свойства класса.

```
public class B
{
    private decimal _seconds; // Поле
    public decimal Seconds // Свойство
    {
        get
        {
            return _seconds;
        }
        protected set
        {
            _seconds = value;
        }
    }

    public decimal Minutes // Свойство
    {
        get
        {
            return _seconds / 60;
        }
        set
        {
            _seconds = value * 60;
        }
    }

    public decimal GetHours() // Метод
    {
        var hours = _seconds / 3600;
        return hours;
    }
    public decimal GetHours2() => _seconds / 3600; // Метод

    public static void ConsolePrint(B b) // Статический метод
    {
        Console.WriteLine($"Секунд: {b.Seconds}");
        Console.WriteLine($"Минут: {b.Minutes}");
        Console.WriteLine($"Часов: {b.GetHours()}");
    }
}
```

```
B b = new B(); // Объект класса (Представитель класса, Instance)

//b.Seconds = 30; // Ошибка: Нет доступа
b.Minutes = 30; // set Свойства Minutes
Console.WriteLine(b.GetHours2()); // Вызов метода

//b.ConsolePrint(b); // Ошибка: Нет доступа
B.ConsolePrint(b); // Вызов статического элемента класса
```

Помимо стандартных элементов класса могут быть и статические элементы, которые будут относиться к самому классу, а не к его представителям. Примером использования статических элементов является использование методов класса Console (Console.WriteLine());

Конструктор и финализатор класса

При создании экземпляра любого класса вызывается специальный метод, называемый конструктором, который может быть использован для задания значений элементов класса. Конструктор представляет собой метод, который не имеет типа возвращаемого значения и имеет имя, такое же, как и имя класса, для которого объявлен данный конструктор. Класс или структура, могут иметь несколько конструкторов, которые должны отличаться между собой либо количеством принимаемых параметров, либо их типом.

```
public class C
{
    private int _x;
    private int _y;
    private readonly int _z; // Поле, недоступное для записи вне объявления и
    конструктора
    private int k;

    public C() // Конструктор без параметров
    {
        _x = 100500;
        _z = -100500;
    }
    public C(int x, int y, int z, int k) // Конструктор с параметрами
    {
        _x = x;
        _y = y;
        _z = z;
        this.k = k; // this.k - k, принадлежащее классу C
    }
    public C(C c) // Конструктор копирования
    {
        _x = c._x;
        _y = c._y;
        _z = c._z;
        k = c.k;
    }
    public void UpdateZ(int z)
    {
        //_z = z; // Ошибка: Поле ReadOnly
    }
    ~C() // Финализатор класса
    {
        Console.WriteLine("Вызван финализатор класса C");
        throw new Exception("Вызван финализатор класса C");
    }
}
```

Если конструктор не будет объявлен в явном виде, то среда сгенерирует конструктор без параметров. За крайне редким исключением конструктору класса должен быть присвоен модификатор доступа public.

В общем случае в C# разрешено присвоение между объектами одного и того же класса. На практике это означает, что мы получим два указателя на один и тот же объект. Для обеспечения создания нового объекта, которому в момент создания были переданы значения данных существующего объекта, но при этом под него выделялась собственная область памяти и в дальнейшем эти два объекта были бы полностью независимыми, необходим конструктор копирования. Единственным формальным параметром конструктора копирования всегда является переменная типа копируемый класс. При наличии конструктора копирования в классе всегда должен быть и обычный конструктор. За выделение и освобождение памяти в языке C# отвечает сборщик мусора. При каждом создании нового объекта среда выполнения выделяет память для объекта из управляемой динамически распределяемой памяти (чаще всего называемой просто кучей). Когда количество доступной для выделения памяти в куче достигает определенного критического значения, сборщик мусора «отыскивает» в ней объекты, которые больше не используются, и освобождает занимаемую ими память.

Финализатор – метод класса, который автоматически вызывается средой выполнения в промежутке времени между моментом, когда объект этого класса был помечен сборщиком мусора, как неиспользуемый, и моментом удаления объекта. Соответственно, мы не можем с полной уверенностью сказать, когда данный метод отработает, в отличие от деструктора в языке C++ (деструктор вызывается при выходе из области видимости объекта класса или при явном освобождении занимаемой памяти). Финализатор в C# представляет собой реализацию метода Finalize(), который нельзя переопределить, поэтому для вызова финализатора используется синтаксис вызова деструктора из C++, и только в момент компиляции компилятор называет его Finalize().

Наследование

В C# допускается простое наследование: каждый класс может иметь только одного предка. Используя наследование, можно создать базовый класс, который определяет характеристики, присущие множеству связанных объектов. Этот класс затем может быть унаследован другими классами с добавлением в каждый из них своих особенностей. Равнозначные термины (пары): базовый класс – класс-наследник; родительский класс – дочерний класс; класс предок – класс-наследник.

Создадим в качестве примера базовый класс для обработки массива, включающий определение массива, его ввод и вывод. К элементам базового класса с атрибутом доступа private нет доступа из классов – наследников, они, таким образом, не наследуются. Поэтому рекомендуют (если нет на этот счет особых соображений) дать элементам базового класса атрибут доступа protected.

```
class MyBaseArray
{
    protected int[] array;

    /// <summary>
    /// Конструктор класса MyBaseArray без параметров
    /// </summary>
    public MyBaseArray()
```

```
{
    Console.WriteLine("Количество элементов = ");
    int size = Convert.ToInt32(Console.ReadLine());
    array = new int[size];
    for(int i = 0; i < size; i++)
    {
        Console.WriteLine("array[" + i + "] = ");
        array[i] = Convert.ToInt32(Console.ReadLine());
    }
}

/// <summary>
/// Конструктор класса MyBaseArray с одним параметром
/// </summary>
/// <param name="size">Размер массива</param>
public MyBaseArray(int size)
{
    array = new int[size];
    for (int i = 0; i < size; i++)
    {
        Console.WriteLine("array[" + i + "] = ");
        array[i] = Convert.ToInt32(Console.ReadLine());
    }
}

/// <summary>
/// Функция вывода массива на экран
/// </summary>
public void output()
{
    Console.WriteLine();
    for (int i = 0; i < array.Length; i++)
    {
        Console.WriteLine("array[" + i + "] = " + array[i]);
    }
}
}
```

На его базе можно построить классы обработки массивов. В нашем случае – нахождение суммы. Класс-наследник включает все данные своего предка (за исключением данных с атрибутом доступа private). Наследуются по общим правилам и индексаторы, и свойства, а также методы перегрузки операторов.

```
class ProcessArray: MyBaseArray // В качестве базового класса выбран класс MyBaseArray
{
    int _q;

    /// <summary>
    /// Конструктор класса ProcessArray (наследника)
    /// </summary>
    public ProcessArray()
    {
        Console.WriteLine("Граница: ");
        _q = Convert.ToInt32(Console.ReadLine());
    }

    /// <summary>
    /// Функция вычисления суммы элементов массива
    /// </summary>
    /// <returns> Сумма элементов массива, больших заданного значения </returns>
    public int Sum()
    {
        int s = 0;
        foreach (var item in array.Where(i => i > _q))
        {

```

```
        s += item;
    }
    return s;
}
}
```

Использование созданных классов

```
class Program
{
    static void Main(string[] args)
    {
        ProcessArray myFirstInheritance = new ProcessArray();
        // Создание экземпляра класса - наследника
        myFirstInheritance.Output(); // Обращение к методу предка
        int mySum = myFirstInheritance.Sum(); // Обращение к собственному методу

        Console.WriteLine("Сумма = " + mySum);
        Console.ReadLine();
    }
}
```

При создании экземпляра класса, имеющего предка, запускаются все конструкторы: в первую очередь конструктор базового класса и затем конструктор класса – наследника. В нашем случае это означает, что будет осуществлен ввод сначала массива и вслед за ним – границы. При наличии большего количества уровней наследования подряд будут запущены конструкторы всех уровней иерархии, начиная с базового.

Если конструкторы не имеют формальных параметров, то при этом никаких проблем не возникает: каждый конструктор независимо от других выполняет свои операторы. Осталось решить вопрос: как обеспечить передачу параметра/ параметров конструктору класса – предка, в нашем случае конструктору 2. Проще всего это выполнить с помощью списка инициализации в конструкторе класса – наследника.

```
/// <summary>
/// Конструктор класса ProcessArray (наследника) с параметром
/// </summary>
/// <param name="size">Фактический параметр для конструктора-предка</param>
/// <param name="q">Граница</param>
public ProcessArray(int size, int q): base(size)
{
    _q = q;
    Console.Write("Граница: " + _q);
}
}
```

Запись `base(size)` означает, что конструктору базового класса в качестве фактического параметра будет передано значение `size`.

```
static void Main(string[] args)
{
    Console.Write("Количество элементов: ");
    int size = Convert.ToInt32(Console.ReadLine());
    Console.Write("Минимальная граница для расчета суммы: ");
    int minLimit = Convert.ToInt32(Console.ReadLine());
    var array = new ProcessArray(size, minLimit);
    array.Output(); // Метод класса MyBaseArray
    // Метод класса ProcessArray
    Console.WriteLine($"Сумма элементов массива, значения которых больше {minLimit}, равна {array.Sum()}");

    Console.ReadLine();
}
```

Лучше всего придерживаться следующего правила: при написании конструктора класса – наследника позаботиться о параметрах непосредственного предка. Таким образом, даже при большом количестве уровней иерархии будет обеспечена согласованная работа конструкторов.

Механизмы запрета наследования и разделения кода

Механизмы запрета наследования

Если в ходе программирования возникает необходимость «блокировки» переопределения (модификатор `virtual` в базовом классе) или наследования той или иной части вашего класса, используйте ключевое слово «sealed». Такая процедура называется «запечатыванием».

Разделяемые элементы кода

В случае возникновения необходимости разделения программного кода в рамках одного пространства имен, можно использовать модификатор «partial». Используется для разделения кода класса, структуры или интерфейса.

Существует несколько ситуаций, когда желательно разделение определения класса.

- При работе над большими проектами распределение класса между различными файлами позволяет нескольким программистам работать с ним одновременно.
- При работе с использованием автоматически создаваемого источника код можно добавлять в класс без повторного создания файла источника. Visual Studio использует этот подход при создании форм Windows Forms, кода оболочки веб-службы и т. д. Можно создать код, который использует эти классы, без необходимости изменения файла, созданного в Visual Studio.

Помимо классов, структур и интерфейсов «дробить» также можно методы (например, объявление метода (сигнатура) и реализация) в пределах одного класса).

```
public class A
{
    private readonly int _field = 100500; // Присваивание в это поле допускается только
    в инициализаторе или конструкторе
    protected virtual int FunctionFirst() => _field; // Expression Body Definition With
    ">="
    protected virtual double FunctionSecond(int x) => (double)x / _field;
}

public partial class B : A // Часть реализации класса B
{
    protected override int FunctionFirst() => base.FunctionFirst() - 500;
}

public partial class B // Часть реализации класса B
{
    protected sealed override double FunctionSecond(int y) => base.FunctionSecond(y) -
    0.5; // Теперь этот метод нельзя переопределить в наследниках
}

public sealed class C : B // Теперь от данного класса нельзя наследоваться
{
    protected override int FunctionFirst() => base.FunctionFirst() - 1;
    // protected override double FunctionSecond(int z) { return 0; } // невозможность
    переопределения
}
// public class D : C { } // Ошибка наследования
```

Ссылки на объекты

C# является языком, требующим строгого соблюдения типа при присваивании. Автоматическое преобразование типов, применяемое при работе с обычными переменными, не распространяется на переменные ссылочного типа: ссылочная

переменная одного класса не может ссылаться на объект другого класса. Исключение: ссылочной переменной базового класса можно присвоить ссылку на любой класс-наследник.

```
public class First
{
    private int _firstField;
    public First(int firstField)
    {
        _firstField = firstField;
    }
    public int FirstField => _firstField;
}
public class Second : First
{
    private int _secondField;
    public Second(int firstField, int secondField) : base(firstField)
    {
        _secondField = secondField;
    }
    public int SecondField => _secondField;
}

class Program
{
    static void Main(string[] args)
    {
        First f1 = new First(25);
        Second s1 = new Second(35, 54);

        First f2;
        f2 = f1; // Допустимо. Переменные одного типа
        Console.WriteLine($"f1.FirstField {f1.FirstField}, f2.FirstField {f2.FirstField}");

        f2 = s1; // Допустимо. Second - наследник First
        Console.WriteLine($"f1.FirstField {f1.FirstField}, f2.FirstField {f2.FirstField}");

        //Console.WriteLine($"f2.SecondField {f2.SecondField}"); // Ошибка. First не имеет SecondField

        Console.ReadLine();
    }
}
```

Возможность доступа к членам класса зависит от типа ссылочной переменной, а не от типа объекта, на который она ссылается. Поэтому закомментированный оператор будет ошибкой. Наследуемый класс «ничего не знает» о членах класса наследника.

Виртуальные методы

Метод, при определении которого присутствует слово `virtual`, называется виртуальным. Каждый класс - наследник может иметь собственную версию виртуального метода, называется это переопределением и обозначается словом `override`. В C# выбор версии виртуального метода осуществляется в соответствии со значением указателя на момент вызова (а не типом указателя). Это делается во время выполнения программы. Указатель во время выполнения программы может указывать на объекты различных классов, поэтому по одному и тому же указателю могут вызываться разные версии виртуального метода. Переопределенные методы обеспечивают поддержку полиморфизма. Полиморфизм позволяет определять в базовом классе методы, которые будут общими для всех наследников, но каждый наследник, в случае необходимости, может иметь их собственные реализации. Естественно, что интерфейсы виртуального метода и всех его версий должны полностью совпадать. Таким образом, применение виртуальных методов позволяет фиксировать интерфейс метода и потом разработать под этот интерфейс новые реализации. Виртуальными могут быть и свойства, и индексы.

```
public class Shape
{
    protected decimal length, height;
    public Shape(decimal length, decimal height)
    {
        this.length = length;
        this.height = height;
    }
    public virtual void PrintArea() // Введение виртуального метода PrintArea()
    {
        Console.WriteLine("Метод будет определен позднее");
    }
    public override string ToString() // Переопределение метода ToString()
    {
        return $"Длина: {length}, Высота: {height}";
    }
}
public class Triangle : Shape
{
    private decimal area;
    public Triangle(decimal length, decimal height) : base(length, height) { }
    public override void PrintArea() // Переопределение метода PrintArea()
    {
        // base.PrintArea(); // Можно вызвать метод базового класса
        area = length * height / 2;
        Console.WriteLine($"Площадь треугольника с основанием {length} и высотой {height}: {area}");
    }
}
public class Rectangle : Shape
{
    private decimal area;
    public Rectangle(decimal length, decimal height) : base(length, height) { }
    public override void PrintArea() // Переопределение метода PrintArea()
    {
        area = length * height;
        Console.WriteLine($"Площадь прямоугольника с основанием {length} и высотой {height}: {area}");
    }
}
```

Указатель shapeX имеет тип Shape, но он может указывать на все наследники класса Shape. Выбор версии виртуального метода зависит от значения указателя на момент вызова, поэтому вызову shapeX.PrintArea(); соответствуют разные версии PrintArea().

```
Shape shape = new Shape(25, 54);
shape.PrintArea();

Triangle triangle = new Triangle(5, 12);
triangle.PrintArea();

Shape shapeX = triangle; // shapeX указывает на объект класса Triangle
shapeX.PrintArea();

Rectangle rectangle = new Rectangle(25, 54);
rectangle.PrintArea();

shapeX = rectangle; // shapeX указывает на объект класса Rectangle
shapeX.PrintArea();

Console.ReadLine();

Console.WriteLine($"shape: {shape}"); // Использование перегрузки ToString()
Console.WriteLine($"triangle: {triangle}");
Console.WriteLine($"rectangle: {rectangle}");
Console.WriteLine($"shapeX: {shapeX}");
Console.ReadLine();
```

Соккрытие членов базового класса новыми членами

При возникновении необходимости создания собственного метода (свойства, индексатора и события) класса с тем же именем, типом возвращаемого значения и списком параметров, можно использовать ключевое слово «new» вместо слова «override».

```
public class MyBase
{
    private readonly int _covid19;
    public MyBase(int cases)
    {
        _covid19 = cases;
    }
    public virtual string Coronavirus()
    {
        return "Всё плохо";
    }
    public virtual int Covid19 // Виртуальное свойство
    {
        get { return _covid19; }
    }
}
public class YourClass : MyBase
{
    private int _covid19Ru;
    public YourClass(int numberOfCases, int numberInRussia) : base(numberOfCases)
    {
        _covid19Ru = numberInRussia;
    }
    public override string Coronavirus() => " Этот ковид ещё существует!"; //
Переопределение метода
    public new int Covid19 // Не переопределение, а создание нового свойства
    {
        get { return _covid19Ru; }
        set { _covid19Ru = value; }
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        DateTime dateOfChecking = new DateTime(2021, 2, 24);
        int coronavirusCasesTotal = 113054211;
        int casesInRussiaTotal = 4200902;

        YourClass covid = new YourClass(coronavirusCasesTotal, casesInRussiaTotal);

        Console.WriteLine($"Ситуация: {((MyBase)covid).Coronavirus()}"); // Обращение к
        базовому методу невозможно. Приведение избыточно
        Console.WriteLine($"Ситуация: {covid.Coronavirus()}");

        Console.WriteLine($"Случаев в мире по состоянию на
        {dateOfChecking.ToString("dd.MM.yyyy")}: {((MyBase)covid).Covid19()}");
        Console.WriteLine($"А в России по состоянию на
        {dateOfChecking.ToString("dd.MM.yyyy")}: {covid.Covid19()}");

        Console.ReadLine();
    }
}
```

Абстрактные методы и классы

В С# существует возможность введения в базовом классе методов, а их реализацию оставить «на потом». Такие методы называют абстрактными. Абстрактный метод автоматически является и виртуальным, но писать это нельзя. Класс, в котором имеется хотя бы один абстрактный метод, тоже называется абстрактным и такой класс может служить только в качестве базового класса. Создать объекты абстрактных классов невозможно, потому что там нет реализации абстрактных методов. Чтобы класс – наследник абстрактного класса не был в свою очередь абстрактным (хотя и это не запрещено), там должны содержаться переопределения всех наследованных абстрактных методов. Указатель на абстрактный класс может указывать на любой класс – наследник.

```
public abstract class Shape // Абстрактный класс
{
    protected decimal length, height;
    public Shape(decimal length, decimal height)
    {
        this.length = length;
        this.height = height;
    }
    public abstract decimal GetArea(); // Реализация не нужна – метод абстрактный,
    // фиксируется лишь интерфейс метода
    public override string ToString()
    {
        return $"Площадь равна {GetArea()}";
    }
}
public class Triangle : Shape
{
    private decimal area;
    public Triangle(decimal length, decimal height) : base(length, height) { }

    public override decimal GetArea() // Определение метода GetArea()
    {
        area = length * height / 2;
        return area;
    }
    public override string ToString()
```

```
{
    return "Треугольник: " + base.ToString();
}
}
public class Rectangle : Shape
{
    private decimal area;
    public Rectangle(decimal length, decimal height) : base(length, height) { }

    public override decimal GetArea() // Определение метода GetArea()
    {
        area = length * height;
        return area;
    }
    public override string ToString()
    {
        return "Прямоугольник: " + base.ToString();
    }
}
class Program
{
    static void Main(string[] args)
    {
        //Shape shape = new Shape(24, 24); // Ошибка. Нельзя создать представителя
        //абстрактного класса

        Shape shape = new Triangle(25, 54);
        Console.WriteLine(shape);

        shape = new Rectangle(25, 54);
        Console.WriteLine(shape);

        Console.ReadLine();
    }
}
```

Интерфейсы

При практическом программировании иногда возникает необходимость определения действий, которые должны выполняться классом, но без уточнения способов их выполнения. Один способ достижения этого был рассмотрен выше – это абстрактные методы. Абстрактный класс содержит интерфейс метода, но не содержит его реализации. В C# это подход расширен введением интерфейсов, которые содержат только интерфейсы методов, без их реализации. Таким образом, можно полностью отделить интерфейс класса от его реализации.

«По соглашению имена всех интерфейсов .NET снабжаются префиксом в виде заглавной буквы I. При создании собственных специальных интерфейсов рекомендуется также следовать этому соглашению.»

Описание интерфейса:

```
interface имя_интерфейса
{
    Тип_возвращаемого_значения имя_1 (параметры);
    Тип_возвращаемого_значения имя_2 {get};
    ...
}
```

Реализация интерфейса должна находиться в классе – наследнике интерфейса. Ограничение единственного предка в C# на интерфейсы не распространяется: класс может иметь только один класс – предок и сколько угодно интерфейсов – предков. Однако класс должен содержать реализации всех методов, которые содержатся в наследованном интерфейсе. Кроме этого, класс может содержать и собственные методы по общим правилам.

```
public interface IShape // Интерфейс
{
    decimal Length { get; } // Интерфейс свойства с блоком get
    decimal Height { get; set; } // Интерфейс полного свойства
    string GetShapeType(); // Интерфейс метода
    decimal GetArea(); // Интерфейс метода
    decimal GetPerimeter(); // Интерфейс метода
}
public class Triangle : Shape, IShape
{
    private decimal _a, _b, _c;
    public Triangle(decimal a, decimal b, decimal c, decimal h) : base(a, h)
    {
        _a = length;
        _b = b;
        _c = c;

        if (!(GetResultTriangleInequality(a, b, c)
            && GetResultTriangleInequality(b, a, c)
            && GetResultTriangleInequality(c, a, b)))
            throw new Exception("Треугольник не может быть создан");
    }

    public decimal Length => length; // Реализация блока get Свойства Length из IShape

    public decimal Height // Реализация Свойства Height из IShape
    {
        get
        {
            return height;
        }
        set
    }
}
```

```

        {
            height = value;
        }
    }

    public decimal GetArea() => length * height / 2; // Реализация метода GetArea() из
IShape
    public decimal GetPerimeter() // Реализация метода GetPerimeter() из IShape
    {
        return _a + _b + _c;
    }
    public string GetShapeType() // Реализация метода GetShapeType() из IShape
    {
        return $"Треугольник ({_a}, {_b}, {_c})";
    }

    private bool GetResultTriangleInequality(decimal a, decimal b, decimal c) //
Приватный метод, не описанный в интерфейсе
    {
        return a < (b + c);
    }
    public decimal GetAreaFromPerimeter() // Метод, не описанный в интерфейсе
    {
        var p = GetPerimeter() / 2;
        var sqrt = p * (p - _a) * (p - _b) * (p - _c);
        return (decimal)Math.Sqrt(Convert.ToDouble(sqrt));
    }
}
public class Rectangle : Shape, IShape
{
    public Rectangle(decimal length, decimal height) : base(length, height) { }
    public decimal Length => length;

    public decimal Height { get => height; set => height = value; }

    public decimal GetArea()
    {
        return length * height;
    }

    public decimal GetPerimeter()
    {
        return 2 * (length + height);
    }

    public string GetShapeType() => $"Прямоугольник ({length}, {height})";
}

class Program
{
    static void Main(string[] args)
    {
        //IShape triangle = new Triangle(1, 2, 3, 4); // Ошибка создания треугольника
        IShape trinagle = new Triangle(5, 4, 3, 2.4m);
        Console.WriteLine($"{trinagle.GetShapeType()}");
        Console.WriteLine($"Периметр: {trinagle.GetPerimeter()}");
        Console.WriteLine($"Площадь: {trinagle.GetArea()}");
        //Console.WriteLine($"Площадь: {trinagle.GetAreaFromPerimeter()}"); // Ошибка
доступа через интерфейсную ссылку

        IShape rectangle = new Rectangle(5, 2.4m);
        Console.WriteLine($"{rectangle.GetShapeType()}");
        Console.WriteLine($"Периметр: {rectangle.GetPerimeter()}");
        Console.WriteLine($"Площадь: {rectangle.GetArea()}");
    }
}

```

```
        Console.ReadLine();  
    }  
}
```

С интерфейсами связана еще одна интересная возможность: интерфейсные ссылки. Можно объявить переменную типа интерфейс, которая затем может ссылаться на любой класс – наследник этого интерфейса. Поэтому главная функция предыдущего примера может иметь следующий вид:

```
class Program  
{  
    static string GetShapeType(IShape shape)  
    {  
        return shape.GetShapeType() + " " + shape.GetType();  
    }  
    static void Main(string[] args)  
    {  
        IShape shapeX; // Интерфейсная ссылка  
        shapeX = new Triangle(5, 4, 3, 2.4m);  
        Console.WriteLine(GetShapeType(shapeX)); // Треугольник (5, 4, 3)  
                                                    //_18_Interfaces.Triangle  
        shapeX = new Rectangle(5, 2.4m);  
        Console.WriteLine(GetShapeType(shapeX)); // Прямоугольник (5, 2.4)  
                                                    //_18_Interfaces.Rectangle  
        Console.ReadLine();  
    }  
}
```

Интерфейсы IEnumerable и IEnumerator

Данный раздел основан материале из книги «Pro C# 7. With .NET and .NET Core, Andrew Troelsen, Philip Japikse».

Создадим класс Radio, который будет иметь всего один метод, отвечающий за «включение» и «выключение» радио.

```
public class Radio  
{  
    public void TurnOn(bool on)  
    {  
        Console.WriteLine(on ? "Jamming..." : "Quiet time...");  
    }  
}
```

Создадим класс Car, в котором необходимо обозначить максимальную скорость автомобиля, при превышении которой двигатель выходит из строя. Необходимо организовать метод увеличения скорости автомобиля, в котором будет происходить проверка выхода автомобиля из строя.

```
public class Car  
{  
    private bool _carIsDead; // True, если пользователь превысил MaxSpeed  
    private Radio _musicBox = new Radio();  
  
    public const int MaxSpeed = 100;  
    public int CurrentSpeed { get; set; } = 0;  
    public string PetName { get; set; } = "";  
  
    public Car() { }  
    public Car(string name, int speed)  
    {  
        CurrentSpeed = speed;  
        PetName = name;  
    }  
}
```

```
    }

    public void CrankTunes(bool state)
    {
        _musicBox.TurnOn(state);
    }

    public void Accelerate(int delta) // Метод прибавления скорости автомобиля
    {
        if (_carIsDead)
            Console.WriteLine($"{PetName} is out of order...");
        else
            CurrentSpeed += delta;

        if (CurrentSpeed >= MaxSpeed)
        {
            _carIsDead = true;
            CurrentSpeed = 0;
            throw new Exception($"{PetName} has overheated!");
        }
        else
            Console.WriteLine($"=> CurrentSpeed = {CurrentSpeed}");
    }
}
```

Создадим класс Garage, который хранит набор объектов Car.

```
public class Garage
{
    protected Car[] _cars = new Car[4];

    public Garage()
    {
        _cars[0] = new Car("Rusty", 30);
        _cars[1] = new Car("Clunker", 55);
        _cars[2] = new Car("Zippy", 30);
        _cars[3] = new Car("Fred", 30);
    }
}
```

В Main создадим объект Garage.

```
Garage garage = new Garage();
foreach(Car c in garage) // Ошибка
{
    Console.WriteLine($"{c.PetName} is going {c.CurrentSpeed}");
}
```

В данном случае компилятор сообщит об ошибке: *//Error CS1579 foreach statement cannot operate on variables of type 'Garage' because 'Garage' does not contain a public instance or extension definition for 'GetEnumerator' 19_IEnumerableAndIEnumerator ... \19_IEnumerableAndIEnumerator\Program.cs*

В классе Garage не реализован метод GetEnumerator(), который формально определен в интерфейсе IEnumerable.

Классы или структуры, которые поддерживают такое поведение, позиционируются как способные представлять вызывающему коду доступ к содержащимся внутри них элементам.

```
public interface IEnumerable
{
    IEnumerator GetEnumerator();
}
```

Метод GetEnumerator() возвращает ссылку на интерфейс IEnumerator, который обеспечивает инфраструктуру, позволяющую вызывающему коду обходить внутренние объекты, содержащиеся в совместимом с IEnumerable контейнере.

```
public interface IEnumerator
```

```
{
    object Current { get; }
    bool MoveNext();
    void Reset();
}
```

В данном случае разработчик может пойти двумя путями:

1. Реализовать свои методы, содержащиеся в интерфейсах;
2. Воспользоваться методами, которые уже существуют в классе Array как реализация интерфейсов IEnumerable и IEnumerator (безопаснее).

```
public class Garage : IEnumerable
{
    private Car[] _cars = new Car[4];

    public Garage()
    {
        _cars[0] = new Car("Rusty", 30);
        _cars[1] = new Car("Clunker", 55);
        _cars[2] = new Car("Zippy", 30);
        _cars[3] = new Car("Fred", 30);
    }

    public IEnumerator GetEnumerator()
    {
        return _cars.GetEnumerator(); // Получение Enumerator класса Array
    }
}
```

Итераторные методы. Ключевое слово «yield»

Данный раздел основан на материале из книги «Pro C# 7. With .NET and .NET Core, Andrew Troelsen, Philip Japikse».

Итератор – это член, который указывает, в каком виде должны возвращаться внутренние элементы контейнера во время обработки в цикле (foreach). Метод может быть определен разными способами:

```
public IEnumerator GetEnumerator()
{
    for (var i = 0; i < _cars.Length; i++)
    {
        yield return _cars[i];
    }
}
```

```
public IEnumerator GetEnumerator()
{
    foreach (var i in _cars)
    {
        yield return i;
    }
}
```

```
public IEnumerator GetEnumerator()
{
    yield return _cars[0];
    // ...
    yield return _cars[3];
}
```

В любом из вышеописанных вариантах используется ключевое слово yield. Оно применяется для указания значения или значений, которые подлежат возвращению конструкцией foreach вызывающему коду. При достижении оператора yield return текущее

месторасположение в контейнере сохраняется и выполнение возобновляется с этого месторасположения, когда итератор вызывается в следующий раз.

Существует возможность создания своего метода, возвращаемого IEnumerable (именованный итератор). Ключевое слово yield может применяться внутри любого метода независимо от его имени.

```
public IEnumerable GetMyEnumerable()
{
    for (var i = _cars.Length; i != 0; i--)
    {
        yield return _cars[i - 1];
    }
}
```

Функциональность операции итерации может быть инкапсулирована в локальный метод.

```
public IEnumerable GetMyEnumerable()
{
    return localMethod();
    IEnumerable localMethod()
    {
        for(var i = _cars.Length; i != 0; i--)
        {
            yield return _cars[i-1];
        }
    }
}
```

В таком случае (и с локальным методом, и без) обращение в foreach может выглядеть следующим образом:

```
foreach(Car c in garage.GetMyEnumerable())
```

Индексаторы

Индексаторы позволяют организовать доступ к элементам одного массива в составе класса через имя класса как для получения значения элемента массива, так и для его изменения. Индексатор может иметь атрибут доступа.

Определение индексатора:

```
Тип_данных_элементов_массива this[int индекс]
{
    get
    {
        // Возврат значения
    }
    set
    {
        // Присвоение значения
    }
}
```

Благодаря индексатору можно писать «Имя_класса[i]» вместо «Имя_класса.массив[i]». Кроме того, индексатор в данном случае имеет атрибут доступа public; если мы хотим обратиться к массиву без индексатора «Имя_класса.массив[i]» то пришлось бы изменить и атрибут доступа массива.

Создадим класс NewGarage, который является наследником Garage, и добавим в него индексатор (без проверки на исключительные ситуации).

```
public class NewGarage : Garage
{
    public int Length => _cars.Length;
    public Car this[int index] // Индексатор
    {
```

```
        get
        {
            return _cars[index];
        }
        set
        {
            _cars[index] = value;
        }
    }
}
```

Тогда из Main будет доступен следующий вариант доступа к элементам коллекции:

```
NewGarage newGarage = new NewGarage();
for(int i = 0; i < newGarage.Length; i++)
{
    Console.WriteLine($"{newGarage[i].PetName} is going {newGarage[i].CurrentSpeed}");
}
```

Отражение. Reflection

Как уже обсуждалось ранее, когда разбиралась инкапсуляция, мы не можем обращаться к закрытым элементам класса (полям, например), помеченным модификатором `private`.

Но, существуют нестандартные способы обращения к данным. Один из таких нестандартных способов использует рефлексию. Рассмотрим на примере использование рефлексии (Reflection). Данный механизм в переводе на русский язык также называется «отражение».

Создадим класс `Student` для работы со студентами. Класс рассчитан на то, что никакие поля не могут изменяться (основная информация о студенте).

```
public class Student
{
    private readonly int? _id;
    private string _name;
    private string _lastName;

    public Student(string name, string lastName)
    {
        _name = name;
        _lastName = lastName;
    }
    public Student(int id, string name, string lastName)
    {
        _id = id;
        _name = name;
        _lastName = lastName;
    }
    public Student(int id, Student student)
    {
        _id = id;
        _name = student._name;
        _lastName = student._lastName;
    }

    public int Id => _id ?? 0;
    public string Name => _name;
    public string LastName => _lastName;

    public override string ToString()
    {
        return $"Номер: {Id}, Имя: {Name}, Фамилия: {LastName}";
    }
}
```

Создадим класс для работы с коллекцией студентов

```
public class StudentCollection
{
    private List<Student> _students;

    public string Name { get; set; }
    public Student TeamLead => _students.FirstOrDefault(st => st.Id == 0);

    public StudentCollection()
    {
        _students = new List<Student>();
    }

    public void Add(Student student)
    {
        int id =+ 11901 + _students.Count;
        _students.Add(new Student(id, student));
    }
    public IEnumerable GetMyEnumerable()
    {
        foreach (var student in _students.OrderBy(st => st.LastName)
                                                .ThenBy(st => st.Name))
            yield return student;
    }

    public Student FindByLastName(string lastName)
    {
        return _students.FirstOrDefault(st => st.LastName.ToLower() ==
lastName.ToLower());
    }
}
```

Заполним коллекцию и выведем ее на экран, используя написанный самостоятельно GetMyEnumerable().

```
StudentCollection developers = new StudentCollection();

developers.Add(new Student("Владислав", "Моисеев"));
developers.Add(new Student("Дмитрий", "Адищев"));
developers.Add(new Student("Ольга", "Глушкова"));
developers.Add(new Student("Артур", "Мацейко"));
developers.Add(new Student("Данила", "Францев"));
developers.Add(new Student("Никита", "Кузьмин"));

developers.Name = "ИДБ-19-01";

Console.WriteLine($"Группа: {developers.Name}");
Console.WriteLine($"Староста: {developers.TeamLead}");
foreach(var student in developers.GetMyEnumerable())
{
    Console.WriteLine(student);
}

Console.WriteLine("=====");
Console.ReadLine();
```

Обратим еще раз внимание на то, что класс Student инкапсулирует в себе данные таким образом, что изменить их не получится.

```
private readonly int? _id;
private string _name;
private string _lastName;

public int Id => _id ?? 0; // Свойство для чтения
public string Name => _name; // Свойство для чтения
public string LastName => _lastName; // Свойство для чтения
```

В практическом программировании нередко возникает необходимость обращаться к элементам класса, которые не доступны «из вне» класса.

Далее описанный код не рекомендуется для использования в практическом программировании. С помощью него просто происходит демонстрация возможностей рефлексии!

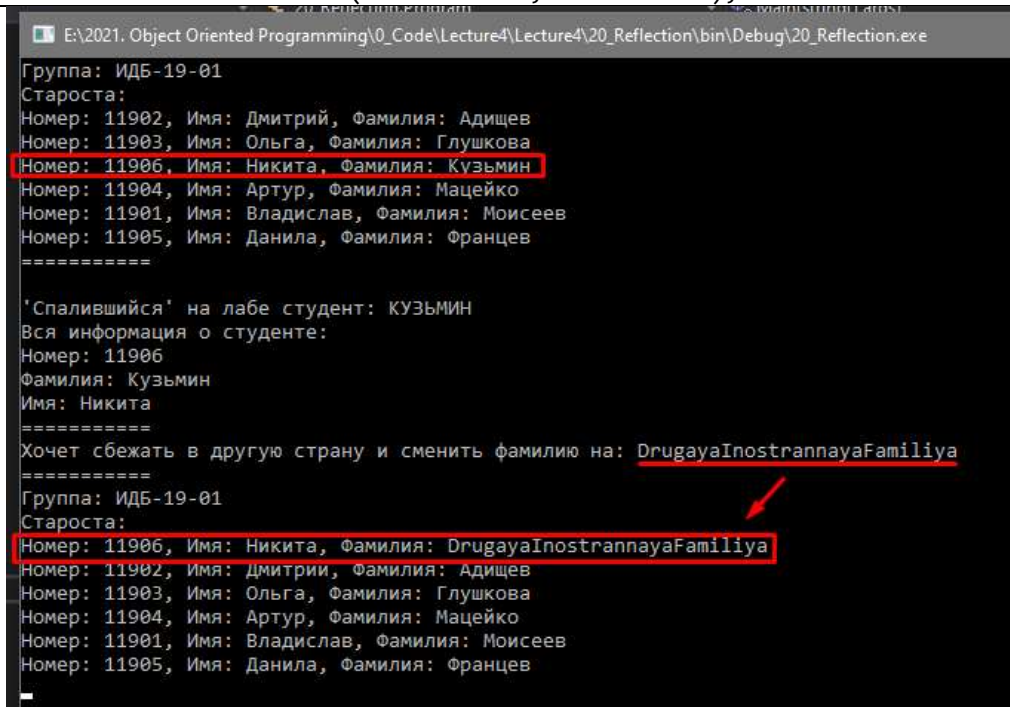
```
Console.WriteLine($"'Спавившийся' на лабе студент: ");
var declassified = developers.FindByLastName(Console.ReadLine());
Console.WriteLine($"Вся информация о студенте:");
Console.WriteLine($"Номер: {declassified.Id}");
Console.WriteLine($"Фамилия: {declassified.LastName}");
Console.WriteLine($"Имя: {declassified.Name}");
Console.WriteLine("=====");

Console.WriteLine($"Хочет сбежать в другую страну и сменить фамилию на: ");
string newLastName = Console.ReadLine();
// declassified.LastName = newLastName; // Ошибка

// Получение типа
var typeOfDeclassified = declassified.GetType();
//var typeOfDeclassified = typeof(Student);

// Получение приватного экземпляра поля
var lastNameForDeclassified = typeOfDeclassified
    .GetField("_lastName",
        BindingFlags.Instance |
        BindingFlags.NonPublic);

// Установка нового значения
lastNameForDeclassified.SetValue(declassified, newLastName);
```



```
E:\2021. Object Oriented Programming\0_Code\Lecture4\Lecture4\20_Reflection\bin\Debug\20_Reflection.exe
Группа: ИДБ-19-01
Староста:
Номер: 11902, Имя: Дмитрий, Фамилия: Адищев
Номер: 11903, Имя: Ольга, Фамилия: Глушкова
Номер: 11906, Имя: Никита, Фамилия: Кузьмин
Номер: 11904, Имя: Артур, Фамилия: Мацейко
Номер: 11901, Имя: Владислав, Фамилия: Моисеев
Номер: 11905, Имя: Данила, Фамилия: Францев
=====
'Спавившийся' на лабе студент: КУЗЬМИН
Вся информация о студенте:
Номер: 11906
Фамилия: Кузьмин
Имя: Никита
=====
Хочет сбежать в другую страну и сменить фамилию на: DrugayaInostrannayaFamiliya
=====
Группа: ИДБ-19-01
Староста:
Номер: 11906, Имя: Никита, Фамилия: DrugayaInostrannayaFamiliya
Номер: 11902, Имя: Дмитрий, Фамилия: Адищев
Номер: 11903, Имя: Ольга, Фамилия: Глушкова
Номер: 11904, Имя: Артур, Фамилия: Мацейко
Номер: 11901, Имя: Владислав, Фамилия: Моисеев
Номер: 11905, Имя: Данила, Фамилия: Францев
```

Рефлексия – это механизм получения объекта типа Type, описывающего тип, модуль или сборку. Так же при помощи рефлексии можно получать значения атрибутов, используемых при создании классов, методов и т.д. Нередко применение рефлексии сводится к тому, чтобы менять поведение программного кода в зависимости от типа данных (модификация программы во время исполнения программного кода).

Класс `Type` является корневым классом для функциональных возможностей рефлексии и основным способом доступа к метаданным.

С помощью членов класса `Type` можно получить сведения об объявленных в типе элементах: конструкторах, методах, полях, свойствах и событиях класса. Помимо этого, с помощью них можно получить информацию о модуле и сборке, в которых развернут данный класс.

При помощи механизма `Reflection` мы можем, например, вывести любые элементы класса, а также их значения.

```
var properties = typeOfDeclassified.GetProperties();
foreach (var prop in properties)
    Console.WriteLine($"{prop.Name}: {prop.GetValue(declassified)}");

Console.ReadLine();
```

```
Id: 11906
Name: Никита
LastName: DrugayaInostrannayaFamiliya
```

В данном случае нам необходимо обратиться к типу объекта для получения перечня элементов (их названий), а для получения значения необходимо передать в метод `GetValue()` объект типа используемого типа.

Типы данных Nullable

Тип данных, отмеченный как nullable – это знаковый тип, который может содержать не только значение, но и NULL (пустая ссылка, которая не ссылается на объект). Ссылочные типы по умолчанию имеют значение NULL.

```
int x = 15;
Console.WriteLine(x);
int? y; // Тип данных Nullable
y = x; // Неявное преобразование nullable -> type

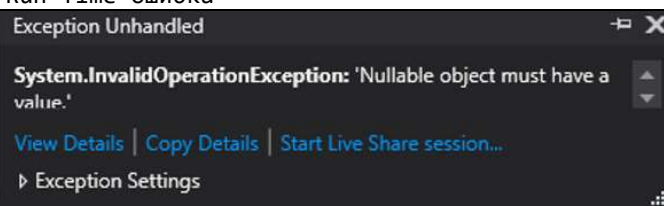
int[] usualArray = new int[3];
usualArray[1] = x;
for (var i = 0; i < usualArray.Length; i++)
{
    Console.WriteLine($"usualArray[{i}] = {usualArray[i]}");
}

int?[] nullableArray = new int?[3];
nullableArray[1] = y;
for (var i = 0; i < nullableArray.Length; i++)
{
    Console.WriteLine($"nullableArray[{i}] = {nullableArray[i]}");
}
```

```
15
usualArray[0] = 0
usualArray[1] = 15
usualArray[2] = 0
nullableArray[0] =
nullableArray[1] = 15
nullableArray[2] =
```

Проверка и получение значения Nullable-переменной

```
//int z = x + y; // Ошибка
int z = x + y.Value;
Console.WriteLine($"z = {z}"); // z = 30;
y = null;
z = x + y.Value; // Run-Time ошибка
```



```
y = null;

z = x;
// 1
if (y.HasValue)
    z += y.Value;
Console.WriteLine($"1) z = {z}"); // 1) z = 15;

// 2
if (y != null)
    z += y.Value;
Console.WriteLine($"2) z = {z}"); // 2) z = 15;

// 3
if (y is int myY)
    z += myY;
Console.WriteLine($"3) z = {z}"); // 3) z = 15;
```

```
// 4
z += y ?? 0;
Console.WriteLine($"4) z = {z}"); // 4) z = 15;
```

Способ 1: через проверку наличия значения (свойство HasValue)

Способ 2: через сравнение с NULL

Способ 3: через оператор is (проверка экземпляра типа и извлечение значения)

Способ 4: через оператор объединения со значением NULL (возвращает значение своего операнда слева, если значение не равно NULL; иначе – возвращает операнд справа).

Кортежи Tuple

Если возникает необходимость группировки нескольких элементов данных в одну структуру данных, но при этом нет необходимости в создании дополнительной структуры данных, можно использовать типы кортежей (Tuple types).

```
// Кортеж
(int, int, string) tuple1 = (0, 24, "Провал");
(int, int, string) tuple2 = (25, 34, "Пойдет");
(int MarkMin, int MarkMax, string Description) tuple3 = (35, 44, "Не плохо");
var tuple4 = (MarkMin: 45, MarkMax: 54, Description: "Класс");
Console.WriteLine(tuple1);
Console.WriteLine(tuple2);
Console.WriteLine(tuple3);
Console.WriteLine(tuple4);

// Использование кортежей без наименований
List<(string, string, int?)> students = new List<(string, string, int?)>();

students.Add(("Иванов", "Иван", 25));
students.Add(("Петров", "Петр", 54));
students.Add(("Васильев", "Василий", null));

students.ForEach(st => Console.WriteLine(st));
students.ForEach(st => Console.WriteLine($"Фамилия: {st.Item1}, Имя: {st.Item2},
Оценка: {st.Item3 ?? 0}"));

// Использование кортежей с именами полей
List<(string LastName, string Name, int? Mark)> students2 = new List<(string, string,
int?)>();

students2.Add(("Иванов", "Иван", 25));
students2.Add(("Петров", "Петр", 54));
students2.Add(("Васильев", "Василий", null));

students2.ForEach(st => Console.WriteLine(st));
students2.ForEach(st => Console.WriteLine($"Фамилия: {st.LastName}, Имя: {st.Name},
Оценка: {st.Mark ?? 0}"));

Console.ReadLine();
```

```
(0, 24, Провал)
(25, 34, Пойдет)
(35, 44, Не плохо)
(45, 54, Класс)

(Иванов, Иван, 25)
(Петров, Петр, 54)
(Васильев, Василий, )
Фамилия: Иванов, Имя: Иван, Оценка: 25
Фамилия: Петров, Имя: Петр, Оценка: 54
Фамилия: Васильев, Имя: Василий, Оценка: 0
(Иванов, Иван, 25)
(Петров, Петр, 54)
(Васильев, Василий, )
Фамилия: Иванов, Имя: Иван, Оценка: 25
Фамилия: Петров, Имя: Петр, Оценка: 54
Фамилия: Васильев, Имя: Василий, Оценка: 0
```

Универсальные шаблоны

Универсальные шаблоны были добавлены в язык C# версии 2.0 и среду CLR. Универсальные шаблоны в платформе .NET Framework представляют концепцию параметров типов, которые позволяют разрабатывать классы и методы, не придерживающиеся спецификации одного или нескольких типов до тех пор, пока класс или метод не будет объявлен клиентским кодом и пока не будет создан его экземпляр. Например, используя параметр универсального типа **T** можно написать отдельный класс, который другой клиентский код сможет использовать без риска приведения во время выполнения или операций упаковки-преобразования.

- Используйте универсальные типы для достижения максимального уровня повторного использования кода, безопасности типа и производительности.
- Наиболее частым случаем использования универсальных шаблонов является создание классов коллекции.
- Библиотека классов платформы .NET Framework содержит несколько новых универсальных классов коллекций в пространстве имен System.Collections.Generic. Их следует использовать по мере возможности вместо таких классов как ArrayList в пространстве имен System.Collections.
- Можно создавать собственные универсальные интерфейсы, классы, методы, события и делегаты.
- Доступ универсальных классов к методам можно ограничить определенными типами данных.
- Сведения о типах, используемых в универсальном типе данных, можно получить во время выполнения путем отражения.

Перегрузка операторов

Цель перегрузки операторов: определение новых правил выполнения существующих операций применительно к созданному пользователем классу. При этом приоритет операций не меняется. Перегрузить можно как бинарные, так и унарные операции, а также операции отношений. Для перегрузки операции необходимо в составе класса задать функцию с новыми правилами выполнения существующей операции; из наиболее

известных операторов нельзя перегрузить оператор присваивания (=), а также составные операторы присваивания (+= и т.п.).

Общий формат перегрузки:

```
public static Тип_возвращаемого_значения operatorЗНАК (TB31 operand1, TB32 operand2)
{
    // ...
}
```

В С#, если имеется перегрузка операции <, то должна быть и перегрузка противоположной операции >. Её отсутствие – синтаксическая ошибка.

Пример (Универсальный шаблон, Индексатор, Перегрузка оператора)

Класс Student из предыдущего примера.

```
public class Helper<T> // T : Использование универсального шаблона
{
    List<T> _myList;
    public int Count => _myList.Count;

    public Helper()
    {
        _myList = new List<T>();
    }

    public T this[int index] // Индексатор
    {
        get
        {
            try
            {
                return _myList[index];
            }
            catch (Exception ex)
            {
                Console.WriteLine(ex.Message);
                return _myList.ElementAtOrDefault(index);
            }
        }
        set
        {
            try
            {
                _myList[index] = value;
            }
            catch (Exception ex)
            {
                Console.WriteLine(ex.Message);
            }
        }
    }

    public void Add(T item)
    {
        _myList.Add(item);
    }

    public void GetList()
    {
        foreach (T item in _myList)
        {
            Console.WriteLine(item);
        }
    }
}
```

```
        public static Helper<T> operator +(Helper<T> operand1, T operand2) // Перегрузка
оператора +
        {
            operand1.Add(operand2);
            return operand1;
        }

        public static Helper<T> operator -(Helper<T> operand1, T operand2) // Перегрузка
оператора -
        {
            operand1._myList.Remove(operand2);
            return operand1;
        }

        public override string ToString()
        {
            string items = "";
            foreach (T item in _myList)
            {
                items += $"{item}\n";
            }
            return items;
        }
    }
}
```

Работа с Helper<T>

```
var ints = new Helper<int>();
ints.Add(10);
ints.Add(15);

Console.WriteLine(ints[0]); // Обращение по индексатору (get)
ints[1] = -15; // Обращение по индексатору (set)
Console.WriteLine("Список:\n" + ints + "Вывод завершен");

ints += 22; // Использование перегрузки
Console.WriteLine("Список:\n" + ints + "Вывод завершен");

Console.ReadLine();

var teamLeads = new Helper<Student>();
teamLeads.Add(new Student("Владислав", "Моисеев"));
teamLeads.Add(new Student("Денис", "Комарков"));
teamLeads.Add(new Student("Никита", "Паничев"));
teamLeads.Add(new Student("Анастасия", "Накленова"));

Console.WriteLine("Старосты второго курса ИТиВС:");
Console.WriteLine(teamLeads);
Console.WriteLine("=====");

teamLeads = teamLeads + new Student("Иван", "Иванов");
Console.WriteLine($"Добавленный студент: {teamLeads[teamLeads.Count - 1]}");
Console.WriteLine("Старосты второго курса ИТиВС:");
Console.WriteLine(teamLeads);
Console.WriteLine("=====");

Console.WriteLine($"Отчисляем студента: {teamLeads[teamLeads.Count - 1]}");
teamLeads -= teamLeads[teamLeads.Count - 1];
Console.WriteLine("Старосты второго курса ИТиВС:");
Console.WriteLine(teamLeads);
Console.WriteLine("=====");

Console.ReadLine();
```

Исключительные ситуации

Исключительная ситуация – это нарушение нормального хода выполнения программы в результате ошибки. Обратим наше внимание на случаи, когда имеющиеся в программе операторы по какой-то причине не могут быть выполнены. Элементарные примеры: деление на нуль, выход индекса за границы. Конечно, появление подобных исключительных ситуаций можно предотвратить путем включения в программу условных операторов проверки корректности, используемых в операции данных. Но их изобилие удлиняет программу и затрудняет восприятие ее логики. Поэтому современный подход к программированию заключается во включении в программу операторов проверки факта возникновения исключительных ситуаций и их обработки. В C# проверка и обработка исключительных ситуаций реализуется операторами `try catch finally`.

```
try
{
    /* Блок кода, подлежащий проверке на наличие ошибок*/
}
catch(Исключительная_ситуация_1 ex0b1)
{
    /* Обработчик для Исключительная_ситуация_1 */
}
catch (Исключительная_ситуация_2 ex0b2)
{
    /* Обработчик для Исключительная_ситуация_2 */
}
catch
{
    /* Обработчик для не идентифицированных исключительных ситуаций*/
}
finally
{
    /* Часть программы, выполняющаяся всегда*/
}
```

В блоке после `try` находятся операторы, проверяемые на наличие исключительной ситуации. Если ни одна исключительная ситуация не возникла, то все блоки `catch` будут пропущены и выполнение программы продолжается с блока `finally`. При возникновении исключительной ситуации выполнение блока `try` прерывается, и начинается поочередное выполнение блоков `catch`. Завершается выполнение блоком `finally`. При выполнении блоков `catch` проверяется по очереди наличие всех перечисленных исключительных ситуаций и выполняются соответствующие блоки. Исключительная ситуация может иметь параметр, тогда при ее обработке можно им пользоваться. Если возникла не перечисленная ни в одном блоке `catch` исключительная ситуация, то выполняется блок без названия ситуации (последний). Завершается выполнение блоком `finally`. Блоки `catch` и `finally` могут присутствовать и независимо друг от друга.

Наиболее распространенные исключительные ситуации приведены в таблице.

Исключение	Значение
<i>ArrayTypeMismatchException</i>	Тип сохраняемого значения несовместим с типом массива
<i>DivideByZeroException</i>	Попытка деления на нуль
<i>IndexOutOfRangeException</i>	Индекс массива оказался вне диапазона
<i>InvalidCastException</i>	Неверно выполнено динамическое приведение типов

<i>OutOfMemoryException</i>	Обращение к оператору new оказалось неудачным из-за недостаточного объема свободной памяти
<i>OverflowException</i>	Имеет место арифметическое переполнение
<i>NullReferenceException</i>	Была сделана попытка использовать нулевую ссылку, т.е. ссылку, которая не указывает ни на какой объект
<i>StackoverflowException</i>	Переполнение стека

Исключение `NullReferenceException` генерируется при попытке использовать нулевую ссылку, например, при попытке вызвать метод, передав ему вместо ссылки на объект нулевую ссылку. Нулевая ссылка не указывает ни на какой объект. Один из способов создать нулевую ссылку — явно присвоить ссылочной переменной `null`-значение, используя ключевое слово `null`.

Генерация исключительных ситуаций

Исключительные ситуации можно создавать самостоятельно.

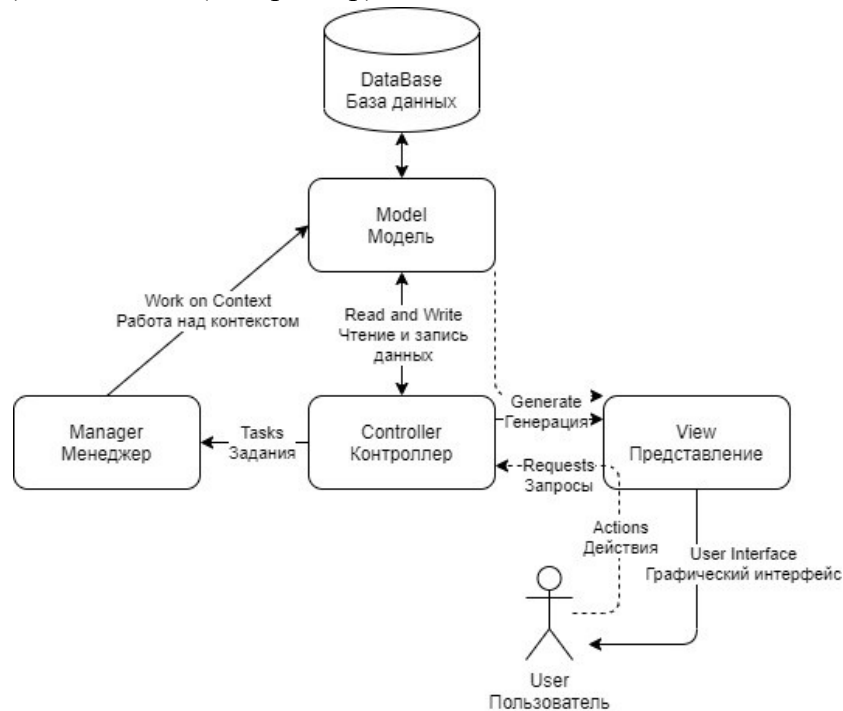
```
public interface ITest
{
    int GetMark();
}
public class Test : ITest
{
    public int GetMark()
    {
        throw new NotImplementedException();
    }
    public int GetSomething()
    {
        throw new Exception("Проблемка!");
    }
}
```

```
var test = new Test();
try
{
    int x = test.GetMark();
    int y = test.GetSomething();
}
catch(Exception ex)
{
    Console.WriteLine(ex);
}
Console.ReadLine();
```

Разработка программного обеспечения

Технология MVC. Model-View-Controller

В практической разработке принято разделять структуру кода таким образом, чтобы было понятно, какая его часть отвечает за данные, какая за обработку этих данных, а какая за графическое представление. Данные компоненты принято называть Model (модель), View (представление) и Controller (контроллер).



Model – это элемент-сущность проекта, View – это внешнее графическое представление для пользователя. Controller отвечает за реакцию проекта на действия пользователя через графический интерфейс и обработку запросов.

Model – часть программы, которая отвечает за работу с базами данных. Как правило, модель представляет собой классы, которые в дальнейшем формируются в коллекции, называемые Context (контекстом). Модель может быть описана при помощи классов языка программирования, используемого в проекте, и, далее, используя ORM (Object-Relational Mapping) связывает их [классы] с таблицами баз данных.

View – часть программы, которая использует язык графической разметки (HTML, CSS, XAML и т.д). Представление позволяет создать пользовательский интерфейс для web-приложения, мобильного устройства любой операционной системы.

Используя структуру MVC, можно разрабатывать приложения, которые работают по принципу «Клиент-Сервер», когда приложение-клиент обращается с запросом на сервер, а тот, в свою очередь, обрабатывает его и возвращает результат.

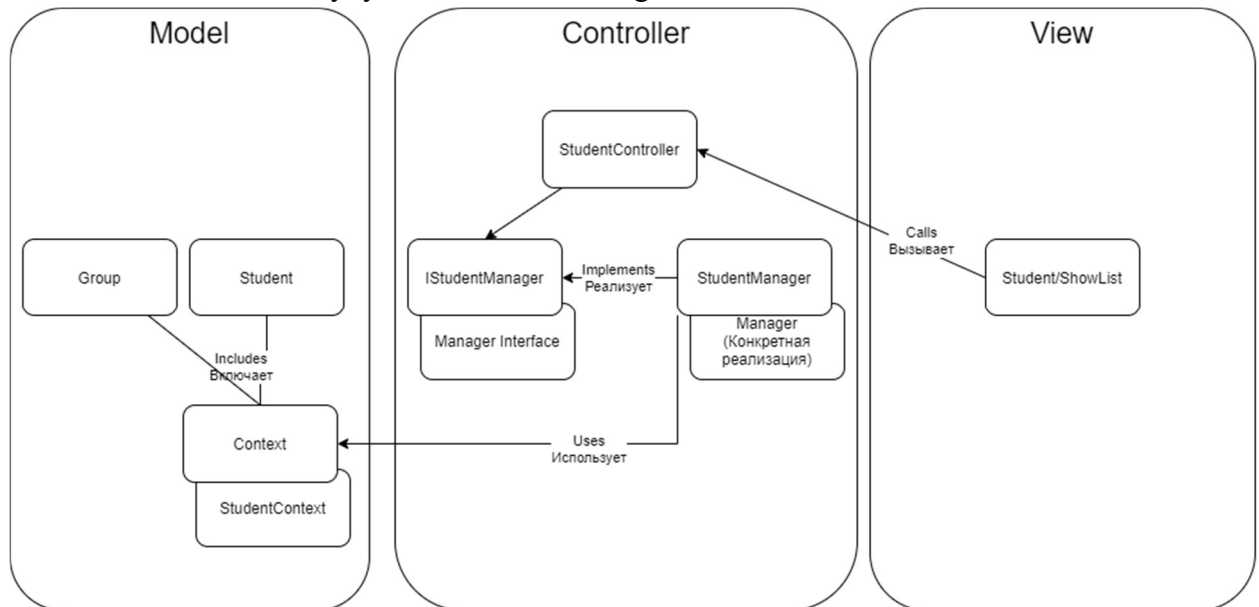
Как описано выше, Controller отвечает за обработку запросов пользователя и внутренних запросов системы.

При работе контроллеров принято называть их именем той сущности, над которой должна производиться работа, с суффиксом «Controller».

Представление должно находиться в папке с наименованием контроллера, а называться так, как называется метод контроллера, который возвращает данное представление.

Manager. Работа с Context

Для создания слабосвязанной архитектуры принято выносить логику работы с данными из контроллера в дополнительные классы, которые реализуют интерфейсы. Таким образом, при изменении бизнес-логики вовсе необязательно менять сам контроллер. Достаточно изменить классы, которые используются внутри контроллера для обработки данных. Такие классы в дальнейшем будут называться Manager'ами.



Слабосвязанная архитектура характеризуется тем, что изменение каких-то отдельных частей программы не будет влиять на изменение всего проекта.

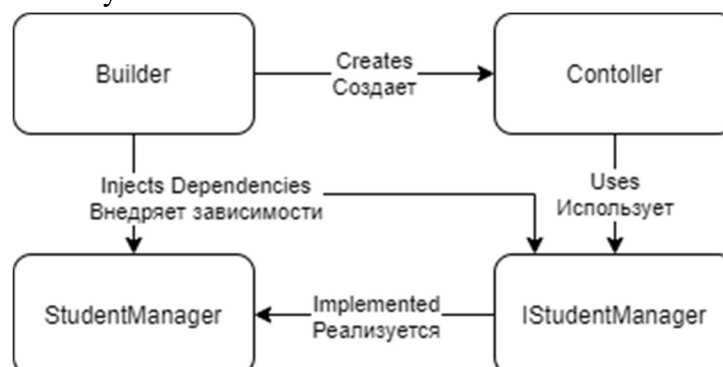
Manager (менеджер) – это класс, который может работать с каким-то контекстом данных. Как правило, называется, как контроллер, но с суффиксом Manager.

Context (контекст) – это класс, который представляет собой структуру базы данных набором свойств коллекций сущностей, расположенных в базе.

Dependency Injection. Внедрение зависимостей в проекте MVC

Для создания слабосвязанного кода используется принцип IoC (Inversion Of Control) в качестве набора рекомендаций. В основе данного принципа лежит то, что каждый компонент системы должен быть как можно более изолированным от других, не полагаясь в своей работе на детали реализации других компонентов.

Dependency Injection – это шаблон, который описывает технику внедрения зависимости программного компоненту.



Способы внедрения зависимостей:

1. Через конструктор;
2. Через свойство;
3. Через интерфейс.

Способы создания зависимостей:

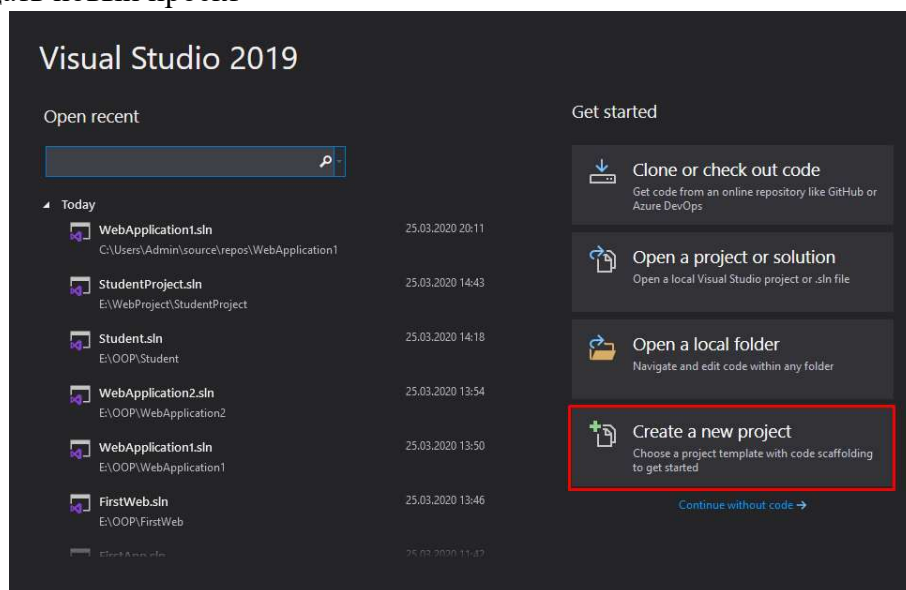
1. С помощью фабрики;
2. С помощью контейнера.

В .NET Core при регистрации зависимости можно указать три варианта времени жизни зависимости:

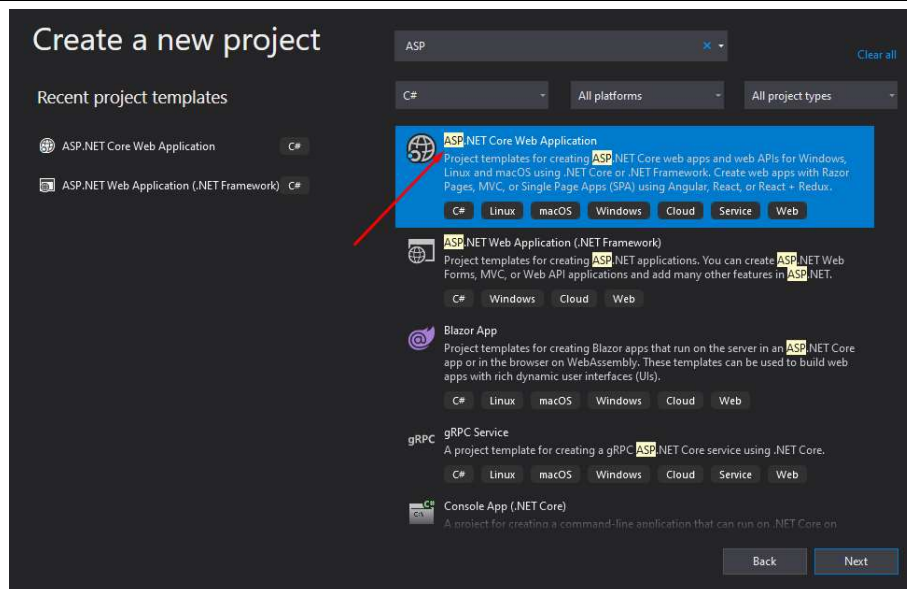
1. Singleton – сервис один на всё приложение;
2. Scoped – сервис создается один раз на запрос;
3. Transient – на каждое требование контейнер создает новый экземпляр сервиса.

Пример создания приложения на ASP.NET Core MVC

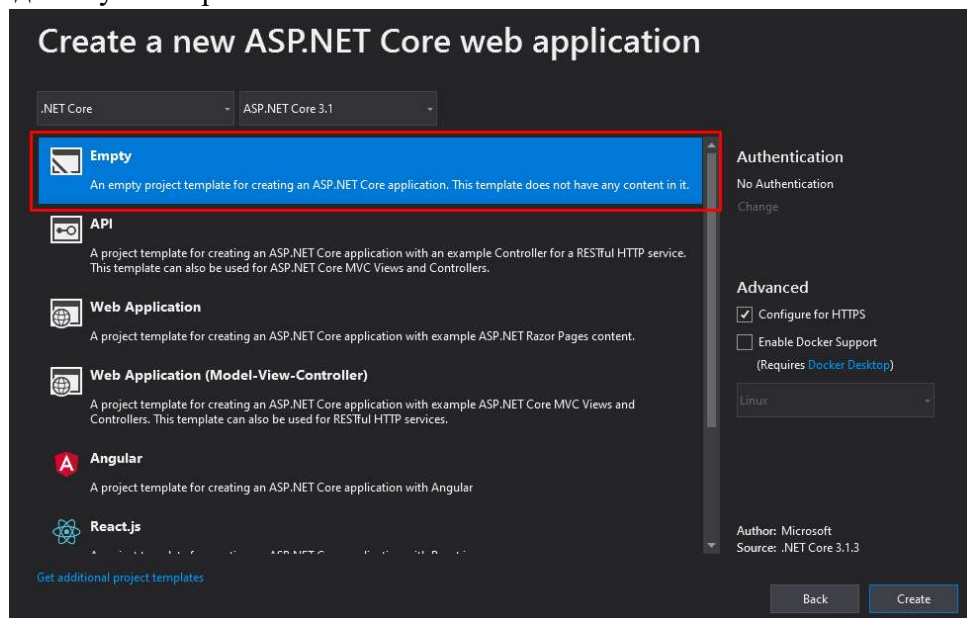
1. Запуск Visual Studio Community 2019.
2. Создать новый проект



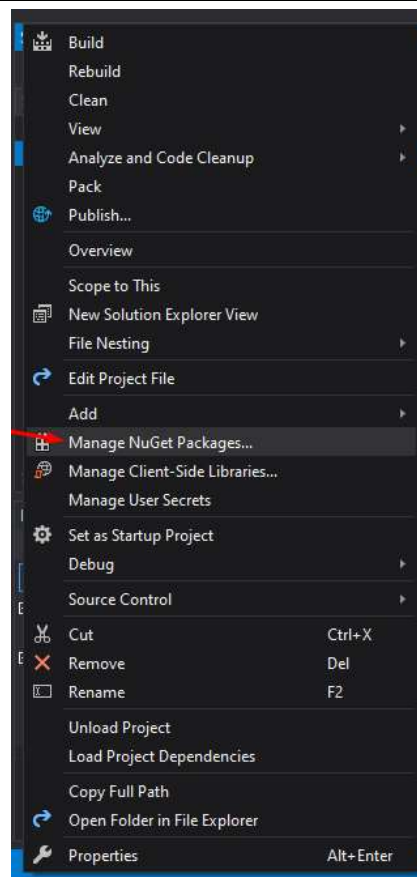
3. Выбрать ASP.NET Core Web Application



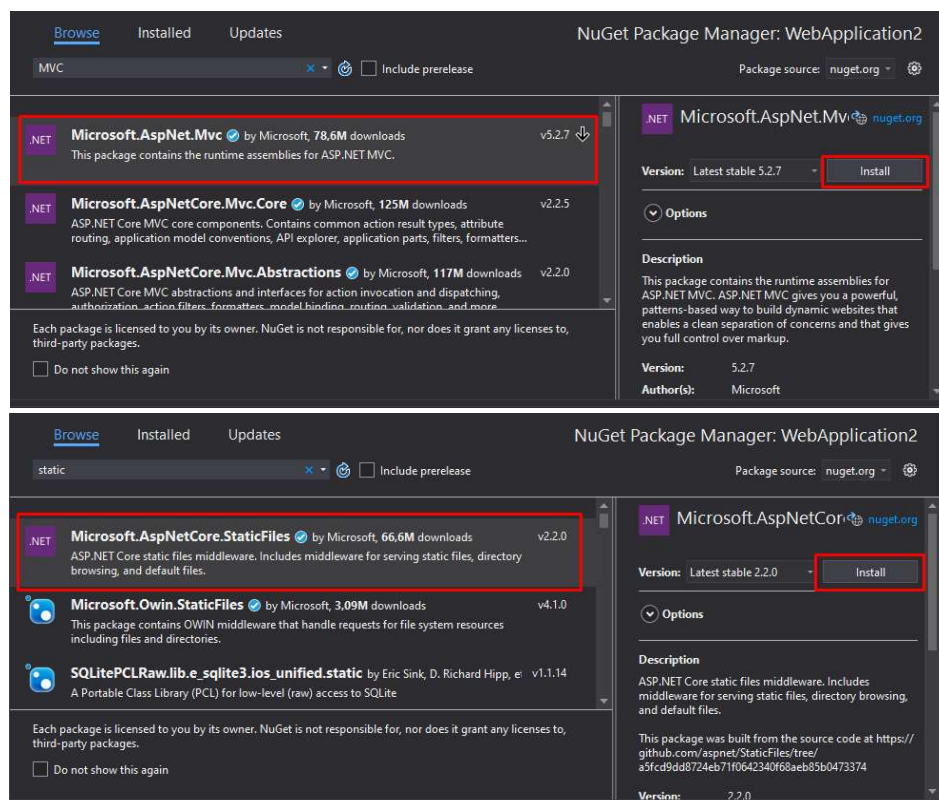
4. Определить расположение файлов проекта и ввести его название
5. Создать пустой проект



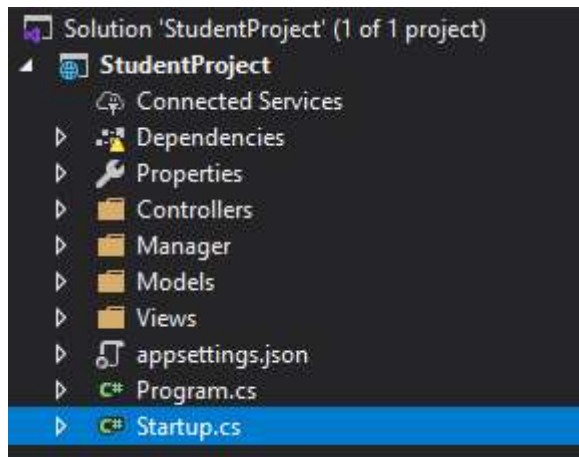
6. Подключить необходимые для работы NuGet-Пакеты. Для этого войти в менеджер, нажав правой кнопкой мыши по названию проекта.



7. В появившемся окне найти (с помощью поиска) необходимые пакеты и установить их



8. Добавить в проект необходимые папки



9. В папку моделей добавляем класс Student

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace StudentProject.Models
{
    public class Student
    {
        public string Name { get; set; }
        public string LastName { get; set; }
        public int Group { get; set; }
        public override string ToString() => $"Студент: {Name} {LastName}, Группа: {Group}";
        public Student(string Name, string LastName, int Group)
        {
            this.Name = Name;
            this.LastName = LastName;
            this.Group = Group;
        }
    }
}
```

10. В папку Manager добавить интерфейс IStudentManager

```
using StudentProject.Models;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace StudentProject.Manager
{
    public interface IStudentManager
    {
        IEnumerable<Student> Students { get; }
        IEnumerable<Student> GetStudentByName(string name);
        void AddStudent(Student student);
    }
}
```

11. Реализовать данный интерфейс в классе Student Manager. Так как в данной лекции не рассматривается использование базы данных, путем добавления статического класса StudentList сделаем «некий контекст» с начальными данными.

```
using StudentProject.Models;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace StudentProject.Manager
```

```
{
    public class StudentManager : IStudentManager
    {
        public StudentManager()
        {
        }

        public IEnumerable<Student> Students => StudentList.students;

        public void AddStudent(Student student)
        {
            StudentList.students.Add(student);
        }

        public IEnumerable<Student> GetStudentByName(string name)
        {
            return StudentList.students.Where(st => st.Name.ToLower() ==
name.ToLower());
        }
    }

    public static class StudentList
    {
        public static List<Student> students = new List<Student>()
        {
            new Student("Юлия", "Попкова", 1801),
            new Student("Александр", "Уколов", 1802),
            new Student("Светлана", "Герасименко", 1803),
            new Student("Анастасия", "Кольцова", 1804));
        }
    }
}
```

12. Изменить файл Startup.cs, дописав в него необходимые вызовы методов

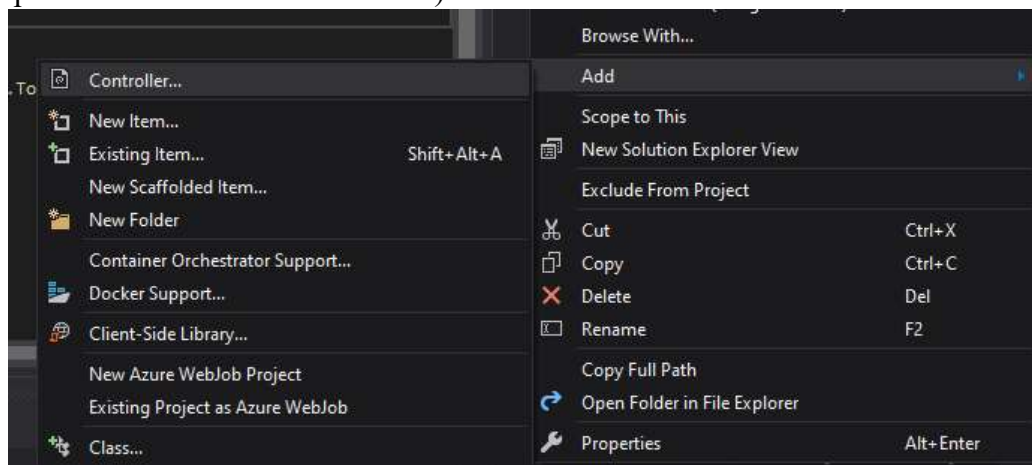
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using StudentProject.Manager;

namespace StudentProject
{
    public class Startup
    {
        // This method gets called by the runtime. Use this method to add services
        // to the container.
        // For more information on how to configure your application, visit
        // https://go.microsoft.com/fwlink/?LinkID=398940
        public void ConfigureServices(IServiceCollection services)
        {
            services.AddMvc(option => option.EnableEndpointRouting = false);
            services.AddTransient<IStudentManager, StudentManager>();
        }

        // This method gets called by the runtime. Use this method to configure the
        // HTTP request pipeline.
    }
}
```

```
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    app.UseDeveloperExceptionPage();
    app.UseStatusCodePages();
    app.UseStaticFiles();
    app.UseMvcWithDefaultRoute();
    app.UseMvc(routes =>
    {
        routes.MapRoute(
            name: "default",
            template: "{controller=Student}/{action=StudentsPage}");
    });
}
}
```

13. В папку Controllers добавить контроллер StudentController путем генерации (нажать правой кнопкой мыши по папке).



14. Создать пустой контроллер. Заменить код контроллера. Ключевое слово «**Controller**» в наименование контроллера – обязательно!

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using StudentProject.Manager;
using StudentProject.Models;

namespace StudentProject.Controllers
{
    public class StudentController : Controller
    {
        private IStudentManager _studentManager;

        public StudentController(IStudentManager studentManager)
        {
            _studentManager = studentManager;
        }

        public IActionResult StudentsPage()
        {
            var students = _studentManager.Students;
            return View(students);
        }

        public IActionResult CreateStudent()
        {

```

```
        return View();
    }

    [HttpPost]
    public ActionResult Create(string Name, string LastName, string Group)
    {
        try
        {
            _studentManager.AddStudent(new Student(Name, LastName,
Convert.ToInt32(Group)));
            return RedirectToAction(nameof(StudentsPage));
        }
        catch
        {
            return RedirectToAction(nameof(StudentsPage));
        }
    }
}
}
```

15. Добавить представление StudentsPage (согласно методу контроллера)

```
@using StudentProject.Models
@model IEnumerable<Student>

@{
    Layout = null;
}

<!DOCTYPE html>

<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>StudentsPage</title>
</head>
<body>
    <h1>Студенты ИТиВС:</h1>
    <table border="1" width="100%" cellpadding="5">
        <tr>
            <th>Фамилия</th>
            <th>Имя</th>
            <th>Группа</th>
        </tr>
        @{
            foreach (var item in Model)
            {
                <tr>
                    <td>@item.LastName</td>
                    <td>@item.Name</td>
                    <td>@item.Group</td>
                </tr>
            }
        }
    </table>

    @using (Html.BeginForm("CreateStudent", "Student"))
    {
        <div>
            <input type="submit" value="Добавить студента" />
        </div>
    }
</body>
```

</html>

16. Добавить представление CreateStudent (согласно методу контроллера)

```
@{
    Layout = null;
}

<!DOCTYPE html>

<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>CreateStudent</title>
</head>
<body>

<h4>Добавление студента</h4>
<hr />
<div class="row">
    <div class="col-md-4">
        @using (Html.BeginForm("Create", "Student"))
        {

            <label>Имя:</label><br />
            <input type="text" name="Name" /><br /><br />
            <label>Фамилия:</label><br />
            <input type="text" name="LastName" /><br /><br />
            <label>Группа:</label><br />
            <input type="text" name="Group" /><br /><br />
            <input type="submit" value="Добавить этого студента" />

        }
    </div>
</div>

<div>
    @Html.ActionLink("Вернуться к списку студентов", "StudentsPage", "Student")
</div>

</body>
</html>
```

17. Запустить проект в любом доступном браузере.

Делегаты

Делегат – это объект, который может ссылаться на метод. Во время выполнения программы один и тот же делегат можно использовать для вызова различных методов, просто заменив метод, на который ссылается этот делегат. Таким образом, метод, который будет вызван делегатом, определяется не в период компиляции программы, а во время ее работы. Делегат в C# соответствует указателю на функцию в C++.

Общий вид объявления делегата:

```
delegate тип_возвращаемого_значения имя_делегата(список_формальных_параметров);
```

Использование делегата:

```
Имя_делегата указатель_на_делегат;  
Указатель_на_делегат = new имя_делегата(имя_функции);
```

Здесь используется только имя функции (параметры не указываются).

Пример 1.

```
delegate int getIntDelegate(); // Объявление делегата

class Test
{
    private int _x;
    private decimal _y;
    public Test(int x, decimal y)
    {
        _x = x;
        _y = y;
    }

    public int GetX()
    {
        return _x;
    }

    public int GetIntY()
    {
        return (int)_y;
    }
}

class Program
{
    static void Main(string[] args)
    {
        var test = new Test(5, -2.5m);

        // Объявление переменной типа делегата с выставлением для нее в соответствие
        метода
        getIntDelegate first = new getIntDelegate(test.GetX);
        getIntDelegate second = new getIntDelegate(test.GetIntY);

        // Использование делегата для вызова метода
        var x = first();
        var y = second();

        Console.WriteLine(x);
        Console.WriteLine(y);

        Console.ReadLine();
    }
}
```

Примечание. В данной функции переменная `second` лишняя, вместо нее можно было использовать `first`.

В этом примере переменная типа делегат лишь заменяет имя метода, при этом фиксировано, к какому объекту относится представляемый метод.

Пример 2. Делегаты позволяют использовать имя метода в качестве формальных параметров.

```
delegate decimal method1(decimal param);

class Example
{
    private decimal[] _array;
    private decimal _a;
    private int _size;

    public Example()
    {
        _size = 5;
        _array = new decimal[_size];

        for(var i = 0; i < _array.Length; i++)
        {
            Console.Write($"Array[{i}] = ");
            _array[i] = Convert.ToDecimal(Console.ReadLine());
        }
    }

    public void MethodUsesDelegateAsParameter(method1 method)
    {
        for(var i = 0; i < _array.Length; i++)
        {
            var item = (decimal)method(_array[i]);
            Console.WriteLine($"Array[{i}] = {item}");
            _a += item;
        }
        Console.WriteLine($"Summ = {_a}");
    }

    public static decimal GetPower2(decimal item)
    {
        return item * item;
    }

    public decimal GetMult2(decimal item)
    {
        return item * 2;
    }
}
```

Делегату `method1` могут соответствовать только функции, имеющие тип возвращаемого значения `decimal` и имеющие один формальный параметр типа `decimal`.

Пример 3. Многоадресный делегат. Одному делегату можно ставить в соответствие несколько функций. В таком случае они будут выполнены в такой последовательности, как они были прикреплены делегату.

```
delegate int MyDeleg(ref string st);

class MyClass
{
    public static int Method1(ref string x)
    {
        Console.WriteLine("I'm Method1");
    }
}
```

```
        x += "1!";  
        return 5;  
    }  
  
    public static int Method2(ref string y)  
    {  
        Console.WriteLine("I'm Method2");  
        y += "2!";  
        return 55;  
    }  
}
```

```
MyDeleg deleg;  
MyDeleg deleg1 = new MyDeleg(MyClass.Method1);  
MyDeleg deleg2 = new MyDeleg(MyClass.Method2);  
  
string myString = "Погнали!";  
  
deleg = deleg1;  
deleg += deleg2;  
  
var k = deleg(ref myString);  
  
Console.WriteLine(myString);  
Console.WriteLine($"k = {k}");  
Console.ReadLine();
```

```
I'm Method1  
I'm Method2  
Погнали!1!2!  
k = 55
```

Делегаты расширяют знакомые нам средства программирования в двух случаях:

- Делегаты как указатели на функцию, что позволяет использовать функции в качестве формальных/фактических параметров других функций.
- Многоадресные делегаты, таким образом получим возможность одним вызовом обеспечить выполнение ряда функций.

Использование делегата в роли псевдонима функции может иногда уменьшить объем наших записей, но не расширяет наши возможности.

Использование делегата Func<>

Делегат Func<T, TResult> имеет следующее определение

```
public delegate TResult Func<in T, out TResult>(T arg);
```

T – тип входного параметра метода, который инкапсулируется данным делегатом;

TResult – тип возвращаемого значения метода, который инкапсулируется данным делегатом.

Для использования делегата Func<,> необходимо написать ключевое слово Func, ввести в угловых скобках через запятую тип входного параметра и тип возвращаемого значения для данного метода, указать наименование метода (который будет использоваться в рамках данного делегата), поставить знак «=», указать имя параметра, который будет использоваться в теле метода, поставить знак «=>», написать тело метода, которое возвратит результат типа TResult.

```
Func<int, int> FibonacciSeq = null;
```

```
FibonacciSeq = index => index > 1 ? FibonacciSeq(index - 1) + FibonacciSeq(index - 2) : index;
```

В данном выражении определен метод `FibonacciSeq`, который вызывается рекурсивно для предыдущих элементов последовательности Фибоначчи.

Метод вызывается обычным способом

```
int item15 = FibonacciSeq(15);
```

Асинхронное программирование

При программировании высоконагруженных систем, web-приложений или систем графического редактирования (например) может возникнуть проблема ожидания отклика программного средства. Для пользователя ожидание отклика программы является критичным фактором. Решением данной проблемы может послужить использование асинхронности в разрабатываемом приложении (использование асинхронных методов или блоков кода). В данном случае такие задачи будут выполняться не в основном потоке.

Для написания асинхронного метода необходимо:

1. При описании метода указать модификатор async (что не является прямым показателем того, что метод будет асинхронным);
2. Данный метод должен возвращать:
 - a. `Task` или `Task<T>`
 - b. `ValueTask<T>` // Структура
 - c. `Void`
3. Метод с модификатором `async` должен содержать хотя бы одно выражение с ключевым словом await.

Пример определения асинхронного метода `FibonacciSequenceAsync()`

```
private void FibonacciSequence()
{
    Func<int, int> FibonacciSeq = null;
    FibonacciSeq = index => index > 1 ? FibonacciSeq(index - 1) + FibonacciSeq(index - 2) : index;

    for (int i = 0; i < _count; i++)
        Console.WriteLine($"Ф_{_name} ({i+1}) = {FibonacciSeq.Invoke(i)}"); //
    Использование Expression при необходимости
    Console.WriteLine(new string('_', 40) + _name + " Завершен");
}

#region Async
private async void FibonacciSequenceAsync()
{
    await Task.Run(() => FibonacciSequence());
}
#endregion
```

В случае необходимости получения результата в асинхронном методе необходимо ключевое слово `await` использовать после знака присваивания или ключевого слова `return`.

Вызов асинхронного метода ничем не отличается от вызова обычного синхронного метода.

Вызов асинхронных операций

Существует два возможных вызова асинхронных операций:

1. Последовательный
2. Параллельный

Последовательный. Несмотря на то, что вызов заданий расположен в асинхронном методе, они будут выполняться в том порядке, в котором описаны в программном коде.

```
private static void GetItemInPower(double item) => Console.WriteLine($"{item} ^ {item} = {Math.Pow(item, item)}");

public async static void GetItemInPowerAsync(double item)
{
    await Task.Run(() => GetItemInPower(item));
    await Task.Run(() => GetItemInPower(item * 2));
    await Task.Run(() => GetItemInPower(item * 3));
}

static void Main(string[] args)
{
    GetItemInPowerAsync(2);
    //2 ^ 2 = 4
    //4 ^ 4 = 256
    //6 ^ 6 = 46656
    Console.ReadLine();
}
```

Параллельный. Для того, чтобы распараллелить выполнение вышеописанных заданий необходимо запустить задачи через статический метод `WhenAll()`, принимающий в качестве параметров массив задач.

```
private static void GetItemInPower(double item) => Console.WriteLine($"{item} ^ {item} = {Math.Pow(item, item)}");

public async static void GetItemInPowerAsync(double item)
{
    await Task.Run(() => GetItemInPower(item));
    await Task.Run(() => GetItemInPower(item * 2));
    await Task.Run(() => GetItemInPower(item * 3));
}

public async static void GetItemInPowerParallelAsync(double item)
{
    Task first = Task.Run(() => GetItemInPower(item));
    Task second = Task.Run(() => GetItemInPower(item * 2));
    Task third = Task.Run(() => GetItemInPower(item * 3));
    await Task.WhenAll(new[] { first, second, third });
}

static void Main(string[] args)
{
    GetItemInPowerParallelAsync(2);
    //6 ^ 6 = 46656    // Результат работы непредсказуем
    //4 ^ 4 = 256
    //2 ^ 2 = 4

    Console.ReadLine();
}
```

В данном случае все три задачи запустятся одновременно. Но, помимо обычных задач создается также задача, которая будет выполнена после завершения всех задач, переданных методу `WhenAll()`.

Введение в Базы Данных Основные понятия

База данных (Database) – это организованная коллекция данных, как правило, хранящаяся и управляемая в электронном виде через компьютерные системы.

Системы управления базами данных (СУБД/ DBMS) – это программное обеспечение, с которым взаимодействует конечный пользователь для доступа к основным возможностям по работе с базами данных.

Реляционные базы данных (Relational Databases) – это базы данных, основанные на таблицах, которые связаны между собой определенным образом. Каждая таблица имеет набор полей любого типа данных и коллекцию значений для них. Как правило, каждая таблица должна иметь хотя бы один уникальный идентификатор (ключ, primary key), по которому можно абсолютно точно найти запись в таблице. Уникальные ключи не повторяются в пределах одной таблицы. Внешний ключ (foreign key) поле в таблице, которое показывает хранит в себе уникальный идентификатор записи из другой таблицы, образуя таким образом связь двух таблиц.

Диаграмма «Сущность-связь». ER-Diagram

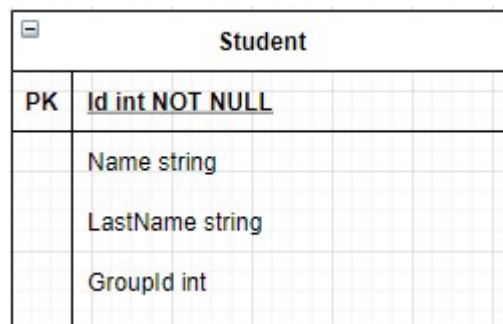
Схема базы данных (Entity-Relationship Diagram (ERD)) – схема, характеризующая таблицы и отношения в этих таблицах. Как правило, в схемах указывается название таблицы, ее поля (первичный ключ, внешние ключи, тип данных, названия полей) и связи.

Возможные связи:

- Один-к-одному (одной записи в таблице можно сопоставить только одну запись из другой таблицы)
- Один-ко-многим (одной записи в таблице может соответствовать несколько записей в другой таблице)
- Многие-ко-многим (нескольким записям одной таблицы соответствует несколько записей другой таблицы)

Обозначения в ER-Диаграмме

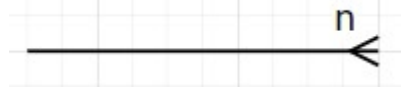
- Сущность. Сущность на ER-диаграмме отображается в виде прямоугольника, в котором отображены названия полей, их типы данных и основные характеристики (not null/ identity и т.д).



- Первичный ключ. Обозначение в виде сокращения от Primary Key – PK.
- Связи (обозначение чисел над связью не является обязательным, но носит рекомендательный характер):
 - В соответствие ставится одна сущность (один) [Основное]



- В соответствие ставится много сущностей (много) [Основное]



- В соответствие ставится хотя бы одна сущность [Дополнительное]



- В соответствие может быть поставлена только одна сущность [Дополнительное]



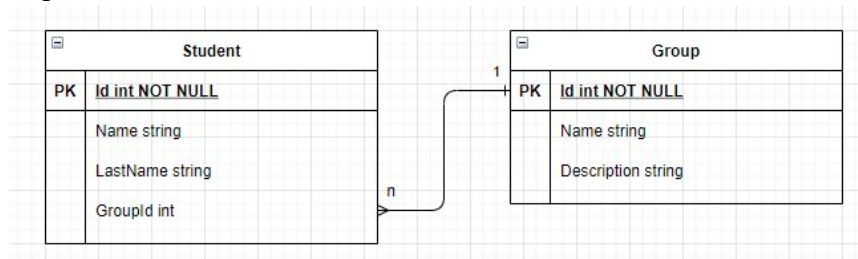
- В соответствие может быть поставлена максимум одна сущность [Дополнительное]



- В соответствие может быть поставлено любое количество сущностей [Дополнительное]



Пример ER-диаграммы со связью «один-ко-многим»



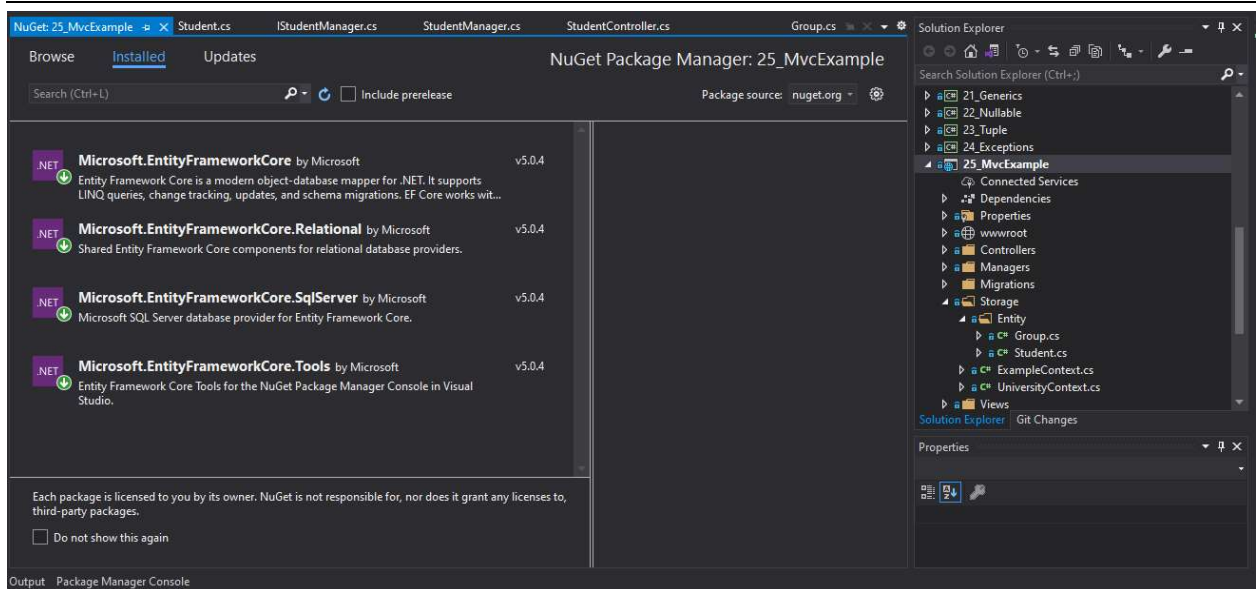
Object-Relational Mapping (ORM). Технология Entity Framework (EF Core)

ORM (Object-Relational Mapping) – механизм, позволяющий использовать Объектно-ориентированный подход к программированию в использовании баз данных.

Примером ORM-технологии является технология Entity Framework Core. Данная технология позволяет использовать классы в виде сущностей и работать с базой данных, как с коллекцией (DbSet) сущностей.

Для работы с технологией EF в проекте необходимо:

1. Скачать и установить все необходимые пакеты (NuGet Packet Manager)



2. Создать классы с помощью автоматически-реализуемых свойств и атрибутов

```
public class Student
{
    [Key] // Введение первичного ключа с помощью атрибута
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)] // Указание на автоматическое
    // заполнение ключа
    public int Id { get; set; }

    [Required]
    public string Name { get; set; }

    [Required]
    public string LastName { get; set; }

    [Required]
    public int GroupId { get; set; }
    [ForeignKey(nameof(GroupId))] // Введение внешнего ключа с помощью атрибута
    public Group Group { get; set; }
}
```

```
public class Group
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string Name { get; set; }
    public string Description { get; set; }
}
```

3. Создать Context по работе с базой данных

```
public class UniversityContext : DbContext // Наследник от класса DbContext
{
    public UniversityContext(DbContextOptions<UniversityContext> options) :
    base(options)
    {
    }

    public DbSet<Student> Students { get; set; } // Коллекция студентов
    public DbSet<Group> Groups { get; set; } // Коллекция групп
}
```

```
}
```

4. В файле Startup необходимо зарегистрировать данный контекст (путь к базе данных и сведения для подключения вводятся в конфигурационных файлах; данный пример служит лишь примером того, как можно зарегистрировать контекст)

```
services.AddDbContext<UniversityContext>(options => options.UseSqlServer(  
    "Server=localhost\\SQLSERVER;Database=UniversityDatabase;Trusted_Connection=True;"));
```

5. Выполнить миграцию по созданию БД через EF путем ввода команды в Package Manager Console

```
Add-Migration InitialCreate
```

6. Внести изменения в БД из полученной миграции

```
Update-Database
```

7. Теперь в Manager можно обращаться к данному контексту (предварительно получив экземпляр контекста из DI) для получения данных

```
public async Task<ICollection<Student>> GetAll()  
{  
    return await _context.Students.Include(st => st.Group).ToListAsync();  
}
```

Обратите внимание на то, что группы необходимо добавить в итоговую выборку при помощи метода Include(), так как данные других сущностей не будут «подтягиваться» автоматически.

8. Пример метода по удалению студента из базы данных

```
public async Task Delete(int id)  
{  
    var student = await _context.Students.FirstOrDefaultAsync(st => st.Id == id);  
    if (student != null)  
        _context.Students.Remove(student);  
    await _context.SaveChangesAsync(); // Сохранение контекста  
}
```

9. Использование языка SQL для получения данных, хранящихся в БД

```
select  
    st.[Id]  
    , st.[Name]  
    , st.[LastName]  
    , gr.[Name]  
    , gr.[Description]  
from      [UniversityDatabase].[dbo].[Students] st  
inner join [UniversityDatabase].[dbo].[Groups] gr  
          on gr.Id = st.GroupId
```

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane shows the 'UniversityDatabase' structure, including tables 'dbo.Groups' and 'dbo.Students'. The 'dbo.Students' table is expanded, showing its columns: 'Id (PK, int, not null)', 'Name (nvarchar(max))', 'LastName (nvarchar(max))', and 'GroupId (FK, int, not null)'. The 'Columns' folder for 'dbo.Students' is selected.

On the right, the 'SQLQuery2.sql' window shows the following query:

```
1 select
2     st.[Id]
3     , st.[Name]
4     , st.[LastName]
5     , gr.[Name]
6     , gr.[Description]
7 from [UniversityDatabase].[dbo].[Students] st
8 inner join [UniversityDatabase].[dbo].[Groups] gr
9     on gr.Id = st.GroupId
```

Below the query window, the 'Results' pane shows the output of the query. The 'Messages' pane indicates 'Query executed successfully.'

	Id	Name	LastName	Name	Description
1	1	Иван	Иванов	ИДБ-19-01	ИТиВС
2	2	Петр	Петров	ИДБ-19-01	ИТиВС
3	3	Аристарх	Сидоров	ИДБ-19-02	ИТиВС
4	5	Ольга	Антонова	ИДБ-19-02	ИТиВС

Принципы Объектно-Ориентированного Проектирования

Принципы Объектно-Ориентированного Проектирования. SOLID Design Principles

SOLID – аббревиатура, полученная из заглавных букв пяти основных принципов объектно-ориентированного проектирования. Эти принципы были введены Робертом Мартином (Robert C. Martin) в его серии книг объектно-ориентированного проектирования «Agile Principles, Patterns, and Practices in C#».

S - Single Responsibility Principle. Принцип единственной ответственности

O - Open-Closed Principle. Принцип открытости-закрытости

L - Liskov Substitution Principle. Принцип подстановки Лисков

I - Interface Segregation Principle. Принцип разделения интерфейса

D - Dependency Inversion Principle. Принцип инверсии зависимостей

Принцип единственной ответственности. Single Responsibility Principle

Данный принцип является одним из лучших советов, как писать программный код. Этот принцип определяет то, что каждый класс имеет всего одну причину изменения.

Вы хотите создать класс Journal, в котором есть коллекция каких-то записей и методы по управлению этих записей:

```
public class Journal
{
    private readonly List<string> entries = new List<string>();
    private static int count = 0;

    public int AddEntry(string text)
    {
        entries.Add($"{++count}: {text}");
        return count; // Memento Pattern (Паттерн Хранитель)
    }

    public void RemoveEntry(int index)
    {
        entries.RemoveAt(index);
    }

    public override string ToString()
    {
        return string.Join(Environment.NewLine, entries);
    }
}

static void Main(string[] args)
{
    var journal = new Journal();
    journal.AddEntry("I am not studying programming today");
    journal.AddEntry("I am just sitting here");
    Console.WriteLine(journal);
    Console.ReadLine();
}
```

Все методы, которые описаны в классе Journal, описывают механизмы по работе с этим журналом.

Предположим, Вы хотите сохранять записи журнала в файле. Некоторые студенты групп, в которых я веду занятия, приступят выполнять эти действия прямо в этом классе Journal, добавляя следующие методы:

```
public void Save(string filename)
```

```
{
    File.WriteAllText(filename, ToString()); // using System.IO;
}

public static Journal Load(string filename)
{
    // ...
    return new Journal();
}

public void Load(Uri uri)
{
}
}
```

Таким образом, принцип Single Responsibility будет нарушен, так как класс Journal теперь отвечает еще и за работу с файловой системой. Это то, чего следует избегать.

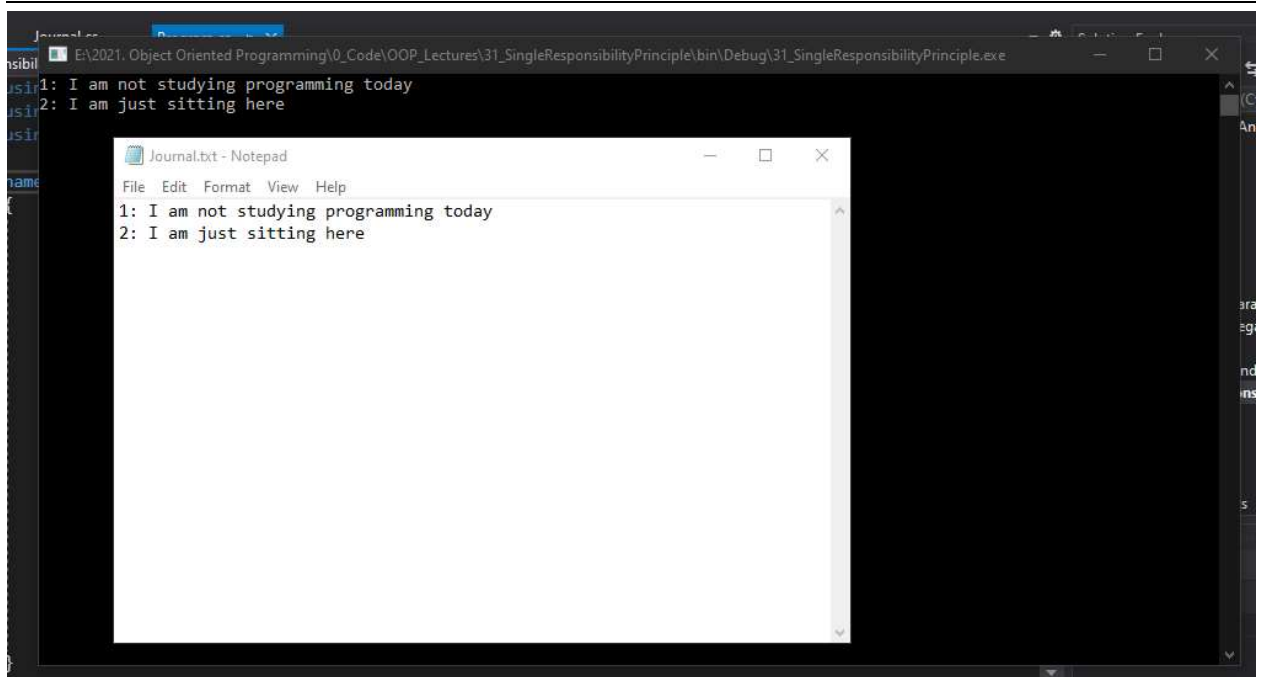
«Лечить» подобного рода ситуации следует при помощи добавления новых классов, которые будут отвечать за что-то конкретное. Например, класс Persistence будет отвечать за работу с файлом:

```
public class Persistence
{
    public void SaveToFile(Journal journal, string filename, bool overwrite = false)
    {
        if (overwrite || !File.Exists(filename))
            File.WriteAllText(filename, journal.ToString());
    }
}

static void Main(string[] args)
{
    var journal = new Journal();
    journal.AddEntry("I am not studying programming today");
    journal.AddEntry("I am just sitting here");
    Console.WriteLine(journal);

    var persistence = new Persistence();
    var filename = @"E:\0_temp\Journal.txt";
    persistence.SaveToFile(journal, filename, true);
    Process.Start(filename); // using System.Diagnostics;

    Console.ReadLine();
}
```



Один класс должен быть ответственным только за одно назначение (инкапсуляция всех необходимых методов в одном классе).

Принцип открытости-закрытости. Open-Closed Principle

Данный принцип лучше объяснять на конкретном примере.

Представьте себе, что Вы разрабатываете web-сайт, где сможете продавать какой-то товар (интернет-магазин). Сами товары могут относиться к разным категориям и иметь разные характеристики, например, цвет и размер:

```
public enum Color
{
    Red,
    Green,
    Blue
}

public enum Size
{
    Small,
    Medium,
    Large,
    Huge
}

public class Product
{
    public string Name; // public fields
    public Color Color;
    public Size Size;

    public Product(string name, Color color, Size size)
    {
        Name = name;
        Color = color;
        Size = size;
    }
}
```

Как и в любом интернет-магазине, пользователь должен иметь возможность фильтровать товары по его характеристикам. Например, заказчик, для которого Вы создаете интернет-магазин, хочет, чтобы там была фильтрация по размеру.

```
public class ProductFilter
{
    public IEnumerable<Product> FilterBySize(IEnumerable<Product> products, Size size)
    {
        foreach (var p in products)
            if (p.Size == size)
                yield return p;
    }
}

static void Main(string[] args)
{
    var apple = new Product("Apple", Color.Green, Size.Small);
    var tree = new Product("Tree", Color.Green, Size.Large);
    var house = new Product("House", Color.Blue, Size.Large);

    Product[] products = { apple, tree, house };
    var filter = new ProductFilter();
    Console.WriteLine("Large Products:");
    foreach (var p in filter.FilterBySize(products, Size.Large))
    {
        Console.WriteLine($"- {p.Name} is Large");
    }
    Console.ReadLine();
}
```

Предположим, заказчик захотел, чтобы была фильтрация и по цвету. Для этого в рамках данного подхода нам необходимо добавить новый метод в класс ProductFilter.

```
public class ProductFilter
{
    public IEnumerable<Product> FilterBySize(IEnumerable<Product> products, Size size)
    {
        foreach (var p in products)
            if (p.Size == size)
                yield return p;
    }

    public IEnumerable<Product> FilterByColor(IEnumerable<Product> products, Color
color)
    {
        foreach (var p in products)
            if (p.Color == color)
                yield return p;
    }
}
```

А теперь заказчик хочет, чтобы в интернет-магазине была возможность фильтрации сразу по двум характеристикам, что предсказуемо. Для этого нам необходимо вернуться в класс ProductFilter и добавить туда новый метод. К сожалению, в данном случае не получится просто добавить новый метод и переименовать в нем элементы.

```
public class ProductFilter
{
    public IEnumerable<Product> FilterBySize(IEnumerable<Product> products, Size size)
    {
        foreach (var p in products)
            if (p.Size == size)
                yield return p;
    }
    public IEnumerable<Product> FilterByColor(IEnumerable<Product> products, Color
color)
    {
        foreach (var p in products)
            if (p.Color == color)
                yield return p;
    }
}
```

```
public IEnumerable<Product> FilterBySizeAndColor(IEnumerable<Product> products,
                                                Size size,
                                                Color color)
{
    foreach (var p in products)
        if (p.Size == size && p.Color == color)
            yield return p;
}
```

И так будет происходить каждый раз, когда нам будет необходимо добавлять новый элемент фильтрации.

Принцип открытости-закрытости говорит о том, что класс должен быть готов к расширению, но он должен быть закрыт для изменения.

Что означает, что никто не должен возвращаться в класс фильтрации и добавлять код, который, по сути, уже там.

Решением данной проблемы является реализация принципа наследования, путем реализации интерфейсов.

Создадим интерфейс ISpecification, который будет отвечать за то, чтобы класс, реализующий этот интерфейс, содержал в себе метод, который бы давал результат того, удовлетворяет ли объект какому-то условию (универсальный шаблон).

```
public interface ISpecification<T>
{
    bool IsSatisfied(T t);
}
```

Создадим интерфейс IFilter, который будет работать с любым типом.

```
public interface IFilter<T>
{
    IEnumerable<T> Filter(IEnumerable<T> items, ISpecification<T> spec);
}
```

Теперь можно реализовать данные интерфейсы, наращивая возможности нашей системы.

```
public class ColorSpecification : ISpecification<Product>
{
    Color color;
    public ColorSpecification(Color color)
    {
        this.color = color;
    }
    public bool IsSatisfied(Product t)
    {
        return t.Color == color;
    }
}

public class BetterFilter : IFilter<Product>
{
    public IEnumerable<Product> Filter(IEnumerable<Product> items,
                                       ISpecification<Product> spec)
    {
        foreach (var i in items)
            if (spec.IsSatisfied(i))
                yield return i;
    }
}
```

Таким образом, работа с фильтрами из Main будет выглядеть так:

```
var filter2 = new BetterFilter();
Console.WriteLine("Green Products:");
foreach(var p in filter2.Filter(products, new ColorSpecification(Color.Green)))
    Console.WriteLine($"- {p.Name} is Green");
```

Создание спецификации для Size:

```
public class SizeSpecification : ISpecification<Product>
{
    Size size;
    public SizeSpecification(Size size)
    {
        this.size = size;
    }
    public bool IsSatisfied(Product t)
    {
        return t.Size == size;
    }
}
```

В данном случае, чтобы расширить функциональность нашей системы, у нас нет необходимости изменять какой-то один класс. Наращивание возможностей системы происходит путем создания новых классов, которые удовлетворяют требованиям реализующихся интерфейсов. Добавим, например, возможность ведения фильтрации по какому-то цвету или размеру одновременно.

```
public class OrSpecification<T> : ISpecification<T>
{
    private ISpecification<T> first, second;
    public OrSpecification(ISpecification<T> first, ISpecification<T> second)
    {
        this.first = first;
        this.second = second;
    }
    public bool IsSatisfied(T t)
    {
        return first.IsSatisfied(t) || second.IsSatisfied(t);
    }
}
```

```
Console.WriteLine("Green or Large Products:");
foreach (var p in filter2.Filter(products,
    new OrSpecification<Product>(new ColorSpecification(Color.Green),
    new SizeSpecification(Size.Large))))
    Console.WriteLine($"- {p.Name} {p.Color} {p.Size}");
```

Принцип подстановки Лисков. Liskov Substitution Principle

Принцип, названный в честь Барбары Лисков (США). Идея данного принципа заключается в том, что у Вас должна быть возможность подставить базовый тип вместо наследуемого. Предположим, у нас есть класс Rectangle, который отвечает за прямоугольник:

```
public class Rectangle
{
    public int Width { get; set; }
    public int Height { get; set; }

    public Rectangle()
    {
    }

    public Rectangle(int width, int height)
    {
        Width = width;
        Height = height;
    }

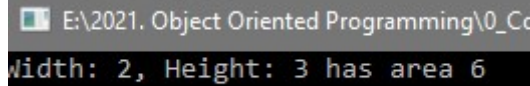
    public override string ToString()
    {
        return $"{nameof(Width)}: {Width}, {nameof(Height)}: {Height}";
    }
}
```

```
}
```

Напишем метод, рассчитывающий площадь прямоугольника, и воспользуемся им:

```
static public int Area(Rectangle r) => r.Width * r.Height;
static void Main(string[] args)
{
    Rectangle rc = new Rectangle(2, 3);
    Console.WriteLine($"{rc} has area {Area(rc)}");

    Console.ReadLine();
}
```



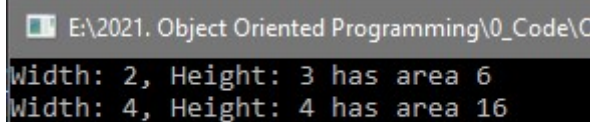
```
E:\2021. Object Oriented Programming\0_Code\01
Width: 2, Height: 3 has area 6
```

Мы знаем, что квадрат является частным случаем прямоугольника. Нам ничто не мешает пользоваться методами прямоугольника, которые были описаны выше. Напишем класс Square.

```
public class Square : Rectangle
{
    public new int Width
    {
        set { base.Width = base.Height = value; }
    }

    public new int Height
    {
        set { base.Width = base.Height = value; }
    }
}
```

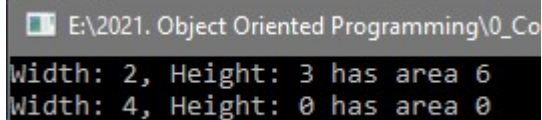
```
Square sq = new Square();
sq.Width = 4;
Console.WriteLine($"{sq} has area {Area(sq)}");
```



```
E:\2021. Object Oriented Programming\0_Code\01
Width: 2, Height: 3 has area 6
Width: 4, Height: 4 has area 16
```

Данный код работает корректно. Давайте, попробуем воспользоваться принципом подстановки Liskov (создадим переменную с типом базового класса):

```
Rectangle sq = new Square();
sq.Width = 4;
Console.WriteLine($"{sq} has area {Area(sq)}");
```



```
E:\2021. Object Oriented Programming\0_Code\01
Width: 2, Height: 3 has area 6
Width: 4, Height: 0 has area 0
```

Обратите внимание, что в данном случае будет ошибка, которая связана с тем, что мы не ввели высоту для класса Rectangle (для класса Square достаточно только ширины).

Исправление данной ситуации приводит нас к принципу полиморфизма и введению в коде виртуальных свойств.

```
public class Rectangle
{
    //public int Width { get; set; }
    //public int Height { get; set; }
    public virtual int Width { get; set; }
    public virtual int Height { get; set; }
```

```
public Rectangle()
{
}

public Rectangle(int width, int height)
{
    Width = width;
    Height = height;
}

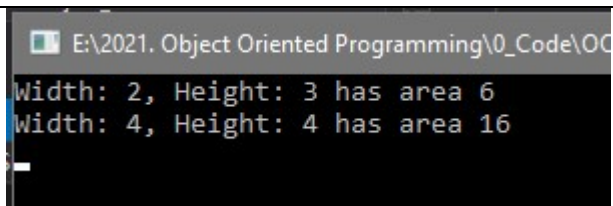
public override string ToString()
{
    return $"{nameof(Width)}: {Width}, {nameof(Height)}: {Height}";
}
}

public class Square : Rectangle
{
    //public new int Width
    //{
    //    set { base.Width = base.Height = value; }
    //}

    //public new int Height
    //{
    //    set { base.Width = base.Height = value; }
    //}

    public override int Width
    {
        set { base.Width = base.Height = value; }
    }

    public override int Height
    {
        set { base.Width = base.Height = value; }
    }
}
```



```
E:\2021. Object Oriented Programming\0_Code\OC
Width: 2, Height: 3 has area 6
Width: 4, Height: 4 has area 16
```

Принцип разделения интерфейса. Interface Segregation Principle

Идея данного принципа состоит в том, чтобы не создавать больших интерфейсов, а пользоваться небольшими. Следствием этого будет являться то, что тем разработчикам, которые будут реализовывать Ваш интерфейс, не нужно реализовывать методы, которые им не будут нужны.

Предположим, у Вас есть класс Document. Вы хотите реализовать много возможностей по работе с документами (копирование, сканирование, печать и так далее), причем эти действия должны выполняться каким-то устройством. Для этого Вы создаете какой-то интерфейс IMachine.

```
public class Document { }  
  
public interface IMachine  
{  
    void Print(Document document);  
    void Copy(Document document);  
    void Scan(Document document);  
    void Fax(Document document);  
}
```

Если Вы будете реализовывать этот интерфейс, Вам придется создавать свое многофункциональное устройство:

```
public class MultiFunctionalPrinter : IMachine  
{  
    public void Copy(Document document)  
    {  
        //  
    }  
  
    public void Fax(Document document)  
    {  
        //  
    }  
  
    public void Print(Document document)  
    {  
        //  
    }  
  
    public void Scan(Document document)  
    {  
        //  
    }  
}
```

Но что будет, если Вы хотите реализовать какой-то старый принтер:

```
public class OldFashionPrinter : IMachine  
{  
    public void Copy(Document document)  
    {  
        throw new NotImplementedException(); // !  
    }  
  
    public void Fax(Document document)  
    {  
        throw new NotImplementedException(); // !  
    }  
  
    public void Print(Document document)  
    {  
        //  
    }  
}
```

```
public void Scan(Document document)
{
    throw new NotImplementedException(); // !
}
```

В данном случае можно генерировать исключительные ситуации, выводить соответствующие сообщения, но интерфейс придется реализовывать полностью.

Избежать такой ситуации поможет «раздробление» большого интерфейса на несколько маленьких.

```
public interface IPrinter
{
    void Print(Document d);
}
```

```
public interface IScanner
{
    void Scan(Document d);
}
```

```
public class Printer : IPrinter
{
    public void Print(Document d){/**/}
}
```

При всем при этом можно реализовывать сразу несколько интерфейсов, «создавая» при этом новые устройства.

```
public class Photocopier : IPrinter, IScanner
{
    public void Print(Document d) {/**/}

    public void Scan(Document d) {/**/}
}
```

Данный принцип не запрещает компоновать интерфейсы, объединяя их в один.

```
public interface IMultiFunctionDevice : IPrinter, IScanner {}
public struct MultiFunctionMachine : IMultiFunctionDevice
{
    // compose this out of several modules
    private IPrinter printer;
    private IScanner scanner;

    public MultiFunctionMachine(IPrinter printer, IScanner scanner)
    {
        this.printer = printer;
        this.scanner = scanner;
    }

    public void Print(Document d)
    {
        printer.Print(d);
    }

    public void Scan(Document d)
    {
        scanner.Scan(d);
    }
}
```

Принцип инверсии зависимостей. Dependency Inversion Principle

Принцип инверсии зависимостей заключается в том, что высокоуровневая часть системы не должна быть зависимой от низкоуровневой части системы напрямую. Вместо этого она должна зависеть от каких-то абстракций.

Давайте попробуем создать генеалогию отношений разных людей. Создадим перечисление связей.

```
public enum Relationship
{
    Parent,
    Child,
    Sibling
    //...
}
```

Создадим, непосредственно, класс, отвечающий за участника этих отношений.

```
public class Person
{
    public string Name { get; set; }
    //public DateTime DateOfBirth { get; set; }
}
```

Низкоуровневой частью данной системы как-раз будет являться механизм, отвечающий за создание взаимосвязей между сущностями.

```
public class Relationships
{
    private List<(Person, Relationship, Person)> _relations =
        new List<(Person, Relationship, Person)>();

    public void AddParentAndChild(Person parent, Person child)
    {
        _relations.Add((parent, Relationship.Parent, child));
        _relations.Add((child, Relationship.Child, parent));
    }
}
```

Для того, чтобы пользоваться данной иерархической БД, создадим дополнительный класс.

```
public class Research
{
    static void Main()
    {
        var parent = new Person() { Name = "Евгений" };
        var child1 = new Person() { Name = "Иван" };
        var child2 = new Person() { Name = "Мария" };

        var relationships = new Relationships();
        relationships.AddParentAndChild(parent, child1);
        relationships.AddParentAndChild(parent, child2);
    }
}
```

Так как наши классы по хранению и обработке разделены, нам необходимо каким-то образом предоставить доступ к обработке коллекции.

```
public class Research
{
    public Research(Relationships relationships)
    {
    }

    static void Main()
    {
        var parent = new Person() { Name = "Евгений" };
        var child1 = new Person() { Name = "Иван" };
        var child2 = new Person() { Name = "Мария" };

        var relationships = new Relationships();
        relationships.AddParentAndChild(parent, child1);
        relationships.AddParentAndChild(parent, child2);

        new Research(relationships);
    }
}
```

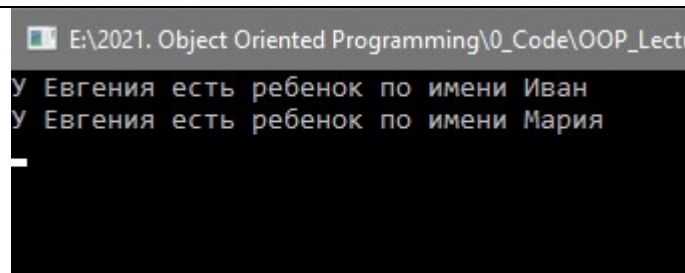
```
}  
}
```

Получение доступа. Первое, что приходит в голову, это открыть доступ для чтения к приватной коллекции в классе Relationships.

```
public List<(Person, Relationship, Person)> Relations => _relations;
```

А затем воспользоваться этой коллекцией.

```
public class Research  
{  
    public Research(Relationships relationships)  
    {  
        var relations = relationships.Relations;  
        foreach(var rel in relations.Where(r =>  
            r.Item1.Name == "Евгений"  
            && r.Item2 == Relationship.Parent))  
            Console.WriteLine($"У Евгения есть ребенок по имени {rel.Item3.Name}");  
        Console.ReadLine();  
    }  
    static void Main()  
    {  
        var parent = new Person() { Name = "Евгений" };  
        var child1 = new Person() { Name = "Иван" };  
        var child2 = new Person() { Name = "Мария" };  
  
        var relationships = new Relationships();  
        relationships.AddParentAndChild(parent, child1);  
        relationships.AddParentAndChild(parent, child2);  
  
        new Research(relationships);  
    }  
}
```



```
E:\2021. Object Oriented Programming\0_Code\OOP_Lect...  
У Евгения есть ребенок по имени Иван  
У Евгения есть ребенок по имени Мария
```

Проблема данного сценария заключается в том, что мы открываем доступ закрытым элементам класса, отвечающего за низкоуровневую часть нашей системы (хранение данных). К тому же мы видим, что в классе Research нам необходимо использовать низкоуровневые части элементов Tuple (Item1, Item2...).

Хорошей идеей для исправления данной ситуации может служить введение дополнительного уровня абстракций. Например, введение интерфейса.

```
public interface IRelationshipBrowser  
{  
    IEnumerable<Person> FindAllChildrenOf(string name);  
}  
  
public class Relationships : IRelationshipBrowser  
{  
    private List<(Person, Relationship, Person)> _relations =  
        new List<(Person, Relationship, Person)>();  
  
    public void AddParentAndChild(Person parent, Person child)  
    {  
        _relations.Add((parent, Relationship.Parent, child));  
        _relations.Add((child, Relationship.Child, parent));  
    }  
}
```

```
public IEnumerable<Person> FindAllChildrenOf(string name)
{
    foreach (var rel in _relations.Where(r =>
        r.Item1.Name == name
        && r.Item2 == Relationship.Parent))
        yield return rel.Item3;
}

//public List<(Person, Relationship, Person)> Relations => _relations;
}
```

В данном случае зависимость происходит уже не на конкретный метод в низкоуровневой части, а на дополнительный уровень абстракции.

```
public class Research
{
    //public Research(Relationships relationships)
    //{
    //    var relations = relationships.Relations;
    //    foreach(var rel in relations.Where(r =>
    //        r.Item1.Name == "Евгений"
    //        && r.Item2 == Relationship.Parent))
    //        Console.WriteLine($"У Евгения есть ребенок по имени {rel.Item3.Name}");
    //    Console.ReadLine();
    //}
    public Research(IRelationshipBrowser browser)
    {
        foreach (var rel in browser.FindAllChildrenOf("Евгений"))
            Console.WriteLine($"У Евгения есть ребенок по имени {rel.Name}");
    }
    static void Main()
    {
        var parent = new Person() { Name = "Евгений" };
        var child1 = new Person() { Name = "Иван" };
        var child2 = new Person() { Name = "Мария" };

        var relationships = new Relationships();
        relationships.AddParentAndChild(parent, child1);
        relationships.AddParentAndChild(parent, child2);

        new Research(relationships);
        Console.ReadLine();
    }
}
```